Black-Box Configuration Search for Generalized Disjunctive Programming

Albert Joon Lee [1], Woosik Kim [2], and David E. Bernal Neira [1,*]

[1]Davidson School of Chemical Engineering, Purdue University, West Lafayette, IN 47907, USA

[2]Elmore Family School of Electrical and Computer Engineering, Purdue University, West Lafayette, IN 47907, USA

*Email: dbernaln@purdue.edu

Abstract

Selecting configurations in generalized disjunctive programming (GDP) can require repeated, costly subproblem evaluations. We compare RBFOpt, Branch-and-Model (BAM), factorization-machine-based optimization with simulated annealing, and a budget-limited discrete steepest descent implementation against sampling controls to examine whether higher target attainment under an evaluation budget translates into lower computational cost. The controlled comparison separates attainment of a reference objective, configuration-evaluation cost, and optimizer overhead using stored objective responses and separately measured subproblem costs. Here we show, across 21 analytical grids and four process GDP benchmark families, that target attainment and computational cost can favor different methods. BAM and a control preserving its fixed sampling prefix attain the same targets at ten evaluations; BAM adds coverage over this control at larger budgets. At 100 evaluations, RBFOpt reaches the reference target in 81.4% of runs compared with 70.0% for BAM and 21.4% for uniform sampling, with median reconstructed search-completion costs of 70.81, 64.42, and 60.51 s, respectively. For the dynamic process benchmark, RBFOpt reaches the target in all ten runs at the smallest tested budget of 12 evaluations, whereas discrete steepest descent first does so at the tested budget of 30. Conventional algebraic methods remain competitive when model expressions and suitable solvers are available. Black-box search completion times are estimated along recorded search paths rather than measured in online optimizer–subproblem executions. The observed trade-offs support reporting target attainment and reconstructed computational cost separately.

Keywords: Generalized disjunctive programming; Black-box optimization; Derivative-free optimization; Surrogate modeling; Process systems engineering; Computational benchmarking.

1 Introduction

In chemical process design, evaluating a single candidate configuration can require a costly optimization or simulation, making the choice of which configuration to explore next a central computational challenge. Process design involves discrete decisions, such as equipment selection, the number of processing stages, and operating modes, along with continuous variables such as flow rates, temperatures, and equipment sizes. Generalized disjunctive programming (GDP) provides a framework for representing these decisions through Boolean variables, logical propositions, and conditional constraints (Raman and Grossmann, 1994). In logic-based GDP solution methods, fixing the Boolean variables associated with a selected configuration determines which constraints remain active and yields a reduced optimization subproblem. When all discrete decisions are fixed, this subproblem can be formulated as a nonlinear programming (NLP) problem. Exploring multiple configurations may therefore require repeatedly solving NLP subproblems, which can be computationally

expensive. Consequently, identifying high-quality configurations with fewer NLP subproblem evaluations is an important objective in solving GDP problems.

Conventional GDP solution methods generally rely on either mixed-integer nonlinear programming (MINLP) reformulations or logic-based decomposition. Big-M and Hull reformulations transform disjunctive constraints into algebraic formulations (Grossmann and Lee, 2003; Raman and Grossmann, 1994), whereas logic-based approaches select discrete configurations and solve the corresponding reduced subproblems (Bergamini et al., 2005; Türkay and Grossmann, 1996). In logic-based approaches, the strategy used to select discrete configurations can significantly affect the number of subproblem evaluations required.

Our previous work introduced Logic-Based Discrete-Steepest Descent (LD-SDA), which represents ordered Boolean decisions using external integer variables (Ovalle et al., 2025). LD-SDA explores the resulting integer space through neighborhood search and discrete line search, evaluating selected configurations by solving their corresponding reduced optimization subproblems. However, as a local search method, LD-SDA may converge to different locally optimal solutions depending on the starting point and the selected neighborhood.

We compare a budget-limited adaptation of LD-SDA, denoted DSDA, with other derivative-free optimization (DFO) methods for searching the same integer lattice. DFO methods use objective evaluations without requiring derivatives with respect to the search variables (Larson et al., 2019). We consider RBFOpt, which uses radial basis function surrogate models (Costa and Nannicini, 2018), and Branch-and-Model (BAM), which combines domain partitioning with local surrogate models (Ma et al., 2023). Factorization machines provide another approach for modeling interactions between decision variables (Rendle, 2010). Kitai et al. (2020) combined factorization machines with quantum annealing in the FMQA framework. Here we test the classical simulated-annealing variant (FMQA-SA). We compare these methods to investigate whether they can identify high-quality solutions with fewer configuration evaluations, while accounting for the computational overhead of each search strategy.

Fewer objective evaluations need not imply lower computational cost (Moré and Wild, 2009), because configuration costs vary and search algorithms incur overhead when selecting new points. Relative to the cited LD-SDA studies, this comparison broadens the configuration-selection strategies and separates reference-target attainment, configuration-evaluation cost, and search overhead. The methods share frozen raw oracle responses, while subsequent sampling, response handling, and stopping remain implementation-specific. The resulting contributions are:

- A paired comparison on 21 analytical Grid-GDP benchmarks and four published process GDP families, sharing the first three evaluated configurations, candidate domains, evaluation-budget accounting, and assessment targets. These experiments compare the four search implementations with uniform sampling and a Grid BAM-prefix control.
- A decomposition of recorded-path costs into configuration evaluation and optimizer overhead, with unsuccessful and repeated evaluations retained. The records identify initialization-driven Grid attainment for BAM, frequent uniform fallback in FMQA-SA, and post-target continuation as mechanisms relevant to the observed rankings. This is a descriptive decomposition of the tested implementations, not a causal ablation of their components.
- A comparison of objective-only search with algebraic GDP implementations that distinguishes target attainment, implementation coverage, and solver-reported certification. Its conclusions are conditional on the tested domains and stopping policies: the Grid points have no remaining original continuous degrees of freedom, and the process searches use two or three external coordinates.

Searches use frozen objective responses; the resulting cost estimates describe the recorded paths rather than measured online optimizer–NLP executions.

2 Background

2.1 Generalized Disjunctive Programming and Fixed-Configuration Evaluation

GDP represents operating and design alternatives using Boolean variables and associated algebraic constraints (Raman and Grossmann, 1994). The original continuous variables have domain $\mathcal{X} = [\underline{\mathbf{x}}, \overline{\mathbf{x}}] \subseteq \mathbb{R}^{n_x}$.

The original integer variables have domain $\mathcal{Z} = [\underline{\mathbf{z}}, \overline{\mathbf{z}}] \cap \mathbb{Z}^{n_z}$. Let K index the disjunctions and D_k index the disjuncts in disjunction $k \in K$. Each disjunct $i \in D_k$ has a Boolean variable Y_{ik}. The GDP formulation is given by

$$\begin{aligned} \min_{\mathbf{x},\mathbf{z},\mathbf{Y}} \quad & f(\mathbf{x},\mathbf{z}) \\ \text{s.t.} \quad & \mathbf{g}(\mathbf{x},\mathbf{z}) \le \mathbf{0}, \\ & \veebar_{i \in D_k} \begin{bmatrix} Y_{ik} \\ \mathbf{h}_{ik}(\mathbf{x},\mathbf{z}) \le \mathbf{0} \end{bmatrix}, \qquad k \in K, \\ & \Omega(\mathbf{Y}) = \text{True}, \\ & \mathbf{x} \in \mathcal{X}, \quad \mathbf{z} \in \mathcal{Z}, \quad \mathbf{Y} \in \{\text{False}, \text{True}\}^{n_y}. \end{aligned} \tag{GDP}$$

Here, f is the objective function, $\mathbf{g}$ represents the global constraints, and $\mathbf{h}_{ik}$ represents the constraints associated with each disjunct. The operator $\veebar$ requires exactly one disjunct to be selected within each disjunction. The logical propositions $\Omega(\mathbf{Y})$ impose additional relationships among the Boolean variables.

Following the external-variable reformulation of Ovalle et al. (2025), each ordered group of m_j Boolean alternatives is represented by an external integer variable $\xi_j \in \{1, \ldots, m_j\}$, which identifies the selected alternative. Let n_E denote the number of external integer variables and $\mathcal{Y}_{\text{logic}}$ the set of Boolean assignments satisfying the exactly-one conditions and $\Omega(\mathbf{Y})$. The external-variable lattice and its logically admissible subset are

$$\begin{aligned} \mathcal{L}_E &= \prod_{j=1}^{n_E} \{1, \ldots, m_j\}, \\ \mathcal{E}_{\text{adm}} &= \{\boldsymbol{\xi} \in \mathcal{L}_E : \mathbf{Y}(\boldsymbol{\xi}) \in \mathcal{Y}_{\text{logic}}\}. \end{aligned} \tag{1}$$

For each logically admissible configuration $\boldsymbol{\xi}$, the corresponding Boolean variables are fixed, yielding the following reduced optimization subproblem:

$$\begin{aligned} \Phi(\boldsymbol{\xi}) = \inf_{\mathbf{x},\mathbf{z}} \quad & f(\mathbf{x},\mathbf{z}) \\ \text{s.t.} \quad & \mathbf{g}(\mathbf{x},\mathbf{z}) \le \mathbf{0}, \\ & \mathbf{h}_{ik}(\mathbf{x},\mathbf{z}) \le \mathbf{0} \quad \text{for all } (i,k) \text{ with } Y_{ik}(\boldsymbol{\xi}) = \text{True}, \\ & \mathbf{x} \in \mathcal{X}, \quad \mathbf{z} \in \mathcal{Z}. \end{aligned} \tag{Sub}$$

Here, $\Phi(\boldsymbol{\xi})$ represents the optimal objective value of the reduced subproblem. For the benchmark problems considered in this study, fixing the Boolean variables yields an NLP. If the reduced subproblem is infeasible, we define $\Phi(\boldsymbol{\xi}) = +\infty$.

When the external-variable mapping represents all logically admissible Boolean configurations, the original GDP can be expressed as the following upper-level optimization problem (Ovalle et al., 2025):

$$\min_{\boldsymbol{\xi} \in \mathcal{E}_{\text{adm}}} \Phi(\boldsymbol{\xi}). \tag{2}$$

A logically admissible configuration may still yield an infeasible reduced subproblem. In this study, candidate configurations are evaluated using precomputed objective values rather than solving the reduced NLP during the search. The NLP evaluation costs are measured separately and used in the computational analysis described in Section 3. Let $v_{\text{lookup}}(\boldsymbol{\xi})$ denote the stored experimental response. For the process cases, a finite response is the objective of an accepted IPOPT solution under the specified initialization and acceptance rules; it is not assumed equal to the theoretical value $\Phi(\boldsymbol{\xi})$. Unaccepted or logically inadmissible configurations retain their outcome status and receive the method-specific responses defined in Section 3. Neither a penalty nor a local-solver termination label proves mathematical infeasibility. The benchmark implementations have no additional nonbinary integer variables after the external choices are specified: Grid choices fix both original coordinates; continuously stirred tank reactor (CSTR) choices fix reactor/bypass and recycle alternatives; Column choices fix the tray configuration; Small Batch choices fix parallel-unit counts; and Dynamic choices fix the stagewise modes. Remaining original decisions are continuous. Supplementary Sections S1 and S5 report the coordinate mappings and unfixed model sizes, respectively (Tables S1 and S5).

2.2 Conventional GDP Solution Approaches

Big-M and Hull reformulations transform GDP models into mixed-integer nonlinear programming (MINLP) formulations by associating Boolean variables Y_{ik} with binary variables y_{ik} and translating logical propositions into algebraic constraints (Grossmann and Lee, 2003; Raman and Grossmann, 1994). In the Big-M reformulation, the disjunctive constraints are expressed as $\mathbf{h}_{ik}(\mathbf{x}, \mathbf{z}) \leq \mathbf{M}_{ik}(1 - y_{ik})$, where the inequality is applied componentwise. The Big-M values must be sufficiently large to render the corresponding constraints redundant when $y_{ik} = 0$, but excessively large values can weaken the continuous relaxation and cause numerical difficulties.

Hull reformulations introduce disaggregated variables, linking constraints, and bounds scaled by binary indicators (Grossmann and Lee, 2003). The disjunctive constraints are imposed on the disaggregated variables when $y_{ik} = 1$, whereas $y_{ik} = 0$ forces these variables to zero, making the corresponding disjunct inactive. For convex disjunctive sets, this formulation can provide a stronger continuous relaxation than Big-M, although it introduces additional variables and constraints. For nonconvex GDP problems, the Hull reformulation does not guarantee a convex reformulated model or global optimality (Chen et al., 2022).

Logic-based outer approximation (LOA) alternates between a mixed-integer master problem and fixed-configuration NLP subproblems (Türkay and Grossmann, 1996). Under suitable convexity and regularity assumptions, the master problem uses linearization cuts to obtain lower bounds, while feasible NLP subproblem solutions provide upper bounds. For nonconvex GDP problems, these linearizations may not provide valid global lower bounds, motivating global logic-based outer approximation (GLOA), which requires globally valid relaxations and subproblem bounds. Bergamini et al. (2005) developed a GLOA approach for process networks using piecewise under- and overestimators.

GDPopt provides implementations of logic-based methods within Pyomo.GDP (Chen et al., 2022; Pyomo Development Team, n.d.). Its relaxation with integer cuts (RIC) method uses an MILP relaxation to select configurations and adds no-good cuts to exclude evaluated discrete assignments, without generating the successive outer-approximation cuts used by LOA (Pyomo Development Team, n.d.).

LD-SDA is an existing alternative that searches ordered external configurations through improving neighborhood moves and discrete line searches (Ovalle et al., 2025). It selects the best improving neighbor, continues along the resulting direction while improvement persists, and resumes neighborhood exploration. The present work uses this discrete-search structure as a reference and compares other configuration-selection strategies and their computational costs.

2.3 Derivative-Free Optimization

Derivative-free optimization (DFO) uses function observations without requiring derivatives of the objective with respect to the search variables (Larson et al., 2019). For GDP configuration search, the outer method can be derivative-free even if its evaluation routine uses a derivative-based nonlinear solver. Surrogate methods seek to reduce expensive evaluations by fitting models to observed data, with computational cost arising from both oracle calls and model-based candidate selection.

2.3.1 RBFOpt

RBFOpt is an open-source surrogate optimizer for bounded black-box problems, with support for continuous, integer, and categorical variables (Costa and Nannicini, 2018; Nannicini, 2021). Given N observed objective values at external integer configurations $\boldsymbol{\xi}^{(1)}, \ldots, \boldsymbol{\xi}^{(N)}$, RBFOpt constructs a surrogate model of the form

$$\widehat{\Phi}_N(\boldsymbol{\xi}) = \sum_{r=1}^{N} \lambda_r \varphi\Big(\|\boldsymbol{\xi} - \boldsymbol{\xi}^{(r)}\|_2 \Big) + p(\boldsymbol{\xi}), \tag{3}$$

where $\widehat{\Phi}_N(\boldsymbol{\xi})$ approximates the objective value at a candidate configuration $\boldsymbol{\xi}$, λ_r are interpolation coefficients determined from the observed values, φ is a radial basis function of the Euclidean distance between configurations, and $p(\boldsymbol{\xi})$ is a polynomial tail depending on the selected kernel.

Candidate selection balances low predicted values against exploration away from evaluated points. The metric stochastic response surface method (MSRSM) combines scaled surrogate predictions and distances

to existing samples, changing their relative weights during the search (Regis and Shoemaker, 2007). New oracle observations update the model.

2.3.2 Branch-and-Model (BAM)

Branch-and-Model (BAM) combines domain partitioning with local surrogate construction (Ma et al., 2023). Evaluated points define boxes, and a potentially optimal selection procedure identifies regions for further modeling and search. Within selected boxes, Automated Learning of Algebraic Models (ALAMO) constructs sparse algebraic models from nearby observations (Cozad et al., 2014). BARON optimizes these local models to propose candidates, while box-size-based exploration maintains broader coverage (Ma et al., 2023; Sahinidis, 1996; The Optimization Firm, 2026). Thus, BARON acts on learned surrogates within BAM; it is not solving the original GDP through this interface. Its support for integer restrictions permits the configuration domains considered here (The Optimization Firm, 2026).

2.4 Factorization-Machine-Based Optimization

A second-order factorization machine (FM) models individual feature effects and pairwise interactions through shared latent parameters (Rendle, 2010). For a binary encoding $\mathbf{b} \in \{0,1\}^m$ of an external integer configuration $\boldsymbol{\xi}$, the fitted FM surrogate is

$$\widehat{\Phi}_{\mathrm{FM}}(\mathbf{b}) = \beta_0 + \sum_{i=1}^{m} \beta_i b_i + \sum_{i=1}^{m-1} \sum_{j=i+1}^{m} \langle \mathbf{a}_i, \mathbf{a}_j \rangle b_i b_j, \tag{4}$$

where $\widehat{\Phi}_{\mathrm{FM}}(\mathbf{b})$ denotes the predicted training response, m is the number of binary features, β_0 is the intercept, β_i is the linear coefficient of b_i, and $\mathbf{a}_i \in \mathbb{R}^r$ is its latent vector of dimension r. The inner product $\langle \mathbf{a}_i, \mathbf{a}_j \rangle$ determines the pairwise interaction coefficient. The model parameters are learned from evaluated configurations and their corresponding training responses.

For binary inputs, $b_i^2 = b_i$, so the fitted model is a binary quadratic model. Dropping its constant term β_0 gives the following quadratic unconstrained binary optimization (QUBO) problem:

$$\min_{\mathbf{b} \in \{0,1\}^m} \mathbf{b}^{\mathsf{T}} \mathbf{Q} \mathbf{b}, \qquad Q_{ij} = \begin{cases} \beta_i, & i = j, \\ \langle \mathbf{a}_i, \mathbf{a}_j \rangle, & i < j, \\ 0, & i > j. \end{cases} \tag{5}$$

The QUBO describes the learned surrogate, not an exact algebraic reformulation of the GDP. Moreover, a binary string need not decode to an in-domain, logically admissible configuration.

The factorization machine with quantum annealing (FMQA) framework combines FM surrogate fitting with annealing-based candidate generation (Kitai et al., 2020). Starting from observed evaluations, the method fits the FM, samples binary candidates from the resulting quadratic model, and evaluates a selected decoded configuration through the objective oracle before updating the training data. In the present experiments, these surrogates are fitted to method-specific responses derived from the precomputed lookup oracle v_{lookup}, rather than certified values of Φ (Section 3).

The simulated annealing variant of FMQA (FMQA-SA) uses classical simulated annealing to sample binary candidates from the learned QUBO. Temperature-dependent acceptance of uphill moves supports exploration, while gradual cooling favors lower-energy solutions (Kirkpatrick et al., 1983). No quantum hardware was used in this study.

3 Experimental Methods

3.1 Benchmark problems: analytical Grid-GDP

We consider 21 two-dimensional analytical functions adapted from the Quantum Hamiltonian Descent benchmark collection (Wu Group, n.d.). Each is sampled at 31 equally spaced values per coordinate, giving 961 configurations. For each benchmark function, let $[\ell_x, u_x]$ and $[\ell_y, u_y]$ denote the coordinate bounds, and

let $I = J = \{0, \ldots, 30\}$ be the corresponding index sets. The Grid optimizers receive $(i, j) \in \{0, \ldots, 30\}^2$ directly. Relative to the one-based convention in Section 2.1, $(\xi_1, \xi_2) = (i + 1, j + 1)$ is only a relabelling of alternatives. FMQA-SA encodes each Grid index in five unsigned bits and rejects decoded values of 31. The physical coordinates are defined as

$$a_i = \ell_x + \frac{i(u_x - \ell_x)}{30}, \qquad b_j = \ell_y + \frac{j(u_y - \ell_y)}{30}. \tag{6}$$

Let f denote the original analytical objective function, and let L and U be its minimum and maximum over the 31×31 lattice. Both constants are obtained by exhaustive preprocessing of all 961 lattice points before any search. A normalized target tolerance of 10^{-5} corresponds to $10^{-5}(U - L)$ in raw objective units, so equal normalized tolerances need not define equally large target sets. Supplementary Section S6, Tables S6–S7, gives L, U, minimizing indices, and target-set sizes for every function. Each benchmark function is represented by the following GDP:

$$\begin{aligned}
\min_{x,y,Y^x,Y^y} \quad & F(x,y) = \frac{f(x,y) - L}{U - L} \\
\text{s.t.} \quad & \bigvee_{i \in I} \begin{bmatrix} Y_i^x \\ x = a_i \end{bmatrix}, \qquad \bigvee_{j \in J} \begin{bmatrix} Y_j^y \\ y = b_j \end{bmatrix}, \\
& \text{ExactlyOne}(Y_0^x, \ldots, Y_{30}^x), \\
& \text{ExactlyOne}(Y_0^y, \ldots, Y_{30}^y), \\
& \ell_x \le x \le u_x, \qquad \ell_y \le y \le u_y, \\
& Y_i^x, Y_j^y \in \{\text{False}, \text{True}\}.
\end{aligned} \tag{7}$$

The two exclusive disjunctions contain 62 coordinate-selection Boolean indicators. The objective retains the analytical expression; it is not a 961-entry lookup-table objective. Consequently, the algebraic solvers receive the expression and its disjunctive structure, whereas the DFO methods observe only objective values returned at their selected coordinates.

Figure 1 shows the discrete Rosenbrock objective landscape. The historical Rosenbrock instance uses

$$f(x, y) = (1 - x)^2 + 100(y - x^2)^2, \qquad x \in [-2, 2], \quad y \in [-1, 3]. \tag{8}$$

On this lattice, $L = 13/2025$ and $U = 2509$ in exact arithmetic. Thus $a_i = -2 + 2i/15$, $b_j = -1 + 2j/15$, and the normalized analytical objective is

$$F(x, y) = \frac{(1 - x)^2 + 100(y - x^2)^2 - 13/2025}{2509 - 13/2025}. \tag{9}$$

The two exactly-one disjunctions in Equation (7) select indices i and j, fixing $x = a_i$ and $y = b_j$; the analytical Rosenbrock objective is then evaluated at that point. The two minimizers have indices $(22, 14)$ and $(23, 16)$, corresponding respectively to $(x, y) = (14/15, 13/15)$ and $(16/15, 17/15)$.

Fixing both coordinate disjunctions fixes x and y, leaving no remaining original continuous degrees of freedom. Thus the measured fixed-point GAMS/IPOPT calls characterize fresh model preparation, solver invocation, and verification, rather than a difficult continuous optimization subproblem. The unfixed analytical objectives remain nonlinear and generally nonconvex, so Big-M and Hull produce MINLP representations. The complete analytical definitions and coordinate domains of the 21 benchmark functions are provided in Supplementary Sections S1–S2 (Table S1) (SECQUOIA, 2026b; Wu Group, n.d.).

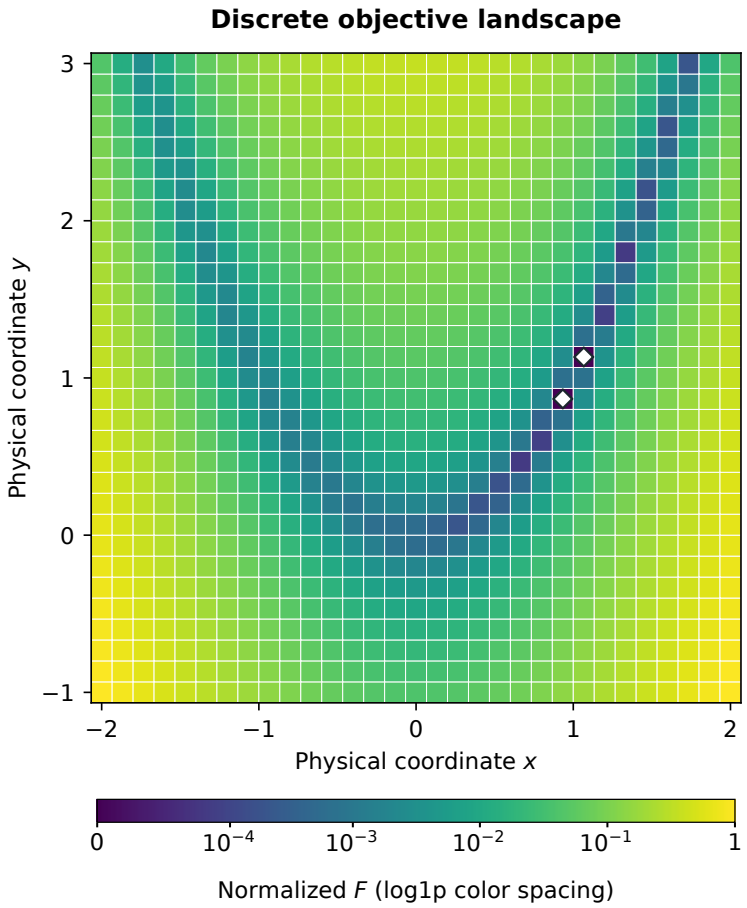


Figure 1: Discrete Rosenbrock objective landscape on the 31×31 grid without interpolation. Colors use $\log_{10}(1+F/10^{-5})$ spacing, with colorbar labels reporting the normalized objective F. White diamonds mark the two lattice minimizers, $(x, y) = (14/15, 13/15)$ and $(16/15, 17/15)$, with grid indices $(i, j) = (22, 14)$ and $(23, 16)$, respectively.

3.2 Process GDP benchmark problems

The process benchmarks are the CSTR network, 17-stage distillation column, and Small Batch models of Ovalle et al. (2025), and the nine-stage nonlinear mode-switching model of Peng et al. (2024). Their objectives minimize total reactor volume (CSTR), condenser and reboiler duties plus active tray cost (Column), investment cost (Small Batch), and the negative integral of squared state (Dynamic). The external variables represent the reactor/feed cutoff and recycle location (CSTR), reflux and boilup positions (Column), parallel unit counts (Small Batch), and final stages of modes 1 and 2 (Dynamic). The CSTR superstructure contains N potential reactor positions, where $N \in \{5, 10, 15, 20, 25, 30\}$. Dynamic uses 30 finite elements per stage and three Radau collocation points. The reference configurations in external variable coordinates are (N, N) for CSTR, $(13, 4)$ for Column (physical trays $(14, 5)$), $(2, 2, 1)$ for Small Batch, and $(1, 4)$ for Dynamic. The cited publications provide the full formulations. CSTR volume is reported in liters and Small Batch investment in the source model's dollars. Column uses the source model's weighted cost proxy based on duties and active trays. Model counts and unit conventions are given in Supplementary Section S5 (Table S5).

Dynamic is an abstract benchmark with no calibrated physical units for state or time. Minimizing its negative squared-state integral favors greater integrated state magnitude and tests mode sequencing coupled to dynamic equations.

Table 1 summarizes the original GDP and DFO candidate domains, IPOPT-accepted lookup coverage, measured evaluation coverage, and evaluation budgets. The process searches use frozen objective tables from fixed-configuration GAMS/IPOPT evaluations; only accepted finite objectives enter the lookup oracle. The objective, acceptance status, and point cost are taken from the same recorded evaluation attempt for the same model, CSTR size, and external-variable tuple. Measured coverage includes all configurations in each DFO candidate box, including unsuccessful and structural-rejection evaluations. The evaluation budget B limits the total number of objective evaluations, including initialization, repeated calls, and unsuccessful evaluations. The full GDP box precedes logical and algebraic feasibility restrictions. Column searches $\{7, \ldots, 15\} \times \{1, \ldots, 9\}$ (81 coordinates) within its original $\{1, \ldots, 15\}^2$ box (225 coordinates). Other cases use their full boxes. The Column search box was selected to limit the scope of the experiment, without a specific engineering or optimization criterion. All DFO methods use this box, and the Column conclusions apply only to it. Configurations outside it are not asserted to be infeasible. For CSTR, recycle/feed ordering admits $N(N + 1)/2$ logical candidates; this does not imply that every candidate produces an accepted IPOPT solution. The CSTR comparisons do not isolate scalability: evaluation budgets change with size, and CSTR30 uses constructor-default NLP initialization whereas sizes 5–25 use mass-balance/zero-recycle initialization. Dynamic permits $z_{E,1} < z_{E,2}$ or $(z_{E,1}, z_{E,2}) = (9, 9)$.

Small Batch uses all 27 configurations in $\{1, 2, 3\}^3$, whose decisions count parallel mixer, reactor, and centrifuge units. Of these configurations, 15 are rejected during feasibility-based bound tightening (FBBT) preprocessing.

Table 1: Process GDP domains, IPOPT-accepted lookup coverage, measured point-cost coverage, evaluation budgets, and reference objective values. Model sizes and unit conventions are given in Supplementary Table S5.

(a) External-variable domains

Model	Full GDP box	DFO candidate box	Full GDP box size	DFO box size
Column	$\{1,\ldots,15\}^2$	$\{7,\ldots,15\}\times\{1,\ldots,9\}$	225	81
CSTR N	$\{1,\ldots,N\}^2$	$\{1,\ldots,N\}^2$	N^2	N^2
Small Batch	$\{1,2,3\}^3$	$\{1,2,3\}^3$	27	27
Dynamic	$\{1,\ldots,9\}\times\{2,\ldots,9\}$	$\{1,\ldots,9\}\times\{2,\ldots,9\}$	72	72

(b) Lookup coverage and evaluation budgets

Model	Finite lookup	Measured coordinates	Evaluation budget (B)	f_{ref}
Column	29	81	16	19346.1248
CSTR 5	15	25	8	3.0620146
CSTR 10	54	100	15	2.8895370
CSTR 15	120	225	22	2.8293941
CSTR 20	210	400	29	2.7994064
CSTR 25	325	625	36	2.7814826
CSTR 30	458	900	43	2.7695697
Small Batch	12	27	10	167427.6571
Dynamic	37	72	12,15,20, 30,37,50	-212.0088
Total	1,260	2,455	–	–

Each measured coordinate is a distinct model/size/coordinate key. The 2,455 keys cover the complete DFO boxes; the 1,790 keys actually visited by the searches are a subset of this inventory.

3.3 Search protocol and compared algorithms

3.3.1 Shared initialization and evaluation budgets

Each Grid-GDP benchmark uses ten initial designs and evaluation budgets $B \in \{10, 30, 50, 70, 100\}$. The primary Grid comparison comprises four search implementations and two sampling controls, giving $21 \times 10 \times 5 \times 6 = 6{,}300$ runs. The process benchmarks comprise 14 model–budget conditions, each evaluated with ten paired seeds, giving 140 runs per method and 700 runs across the four search implementations and uniform sampling (Table 1). For each problem–budget–seed combination, three distinct initial configurations are generated using seeded Latin hypercube sampling, mapped to integer coordinates, and shared in the same order across all methods and sampling controls. This common design controls the first three evaluations; subsequent sampling, response handling, and stopping rules follow the individual implementations. All process cases use seeds 0–9, with no replacement of failed starts. Column seed 0 has three unaccepted initial configurations; the resulting DSDA execution failure remains in the ten-run denominator.

Methods may terminate before exhausting the evaluation budget B. Table 2 separates algorithm stopping from the worker watchdog; detailed recorded options, seeds, versions, and cache behavior are supplied in Supplementary Section S4 and its machine-readable companion files. Internal cache accesses without objective callbacks are not counted as evaluations. Feasibility and target attainment are determined from the original lookup values rather than method-specific penalties. For CSTR, unaccepted evaluations return $+\infty$ to DSDA and RBFOpt; for Column, Dynamic, and Small Batch, those two methods use 10^6. BAM receives `NaN` in all process cases. FMQA-SA retains infinite raw values and separately reports 10^6. These penalties do not replace the recorded solver status or establish mathematical infeasibility.

Table 2: Configuration summary for the compared DFO implementations

Method	Stopping and target information	Candidate generation / seed
DSDA	Budget or no improving move. Grid also stops at the known target and attempts ranked initial starts; process uses one best accepted start.	Fixed L_∞ order, first tie, improvement $> 10^{-12}$; common design seed.
RBFOpt	Budget and native iteration/stall limits; no benchmark target supplied. Native clock limit 10^{30} s (effectively disabled).	MSRSM; genetic candidate search; other Bonmin/IPOPT surrogate solves retained; job seed.
BAM	Native remaining budget $B - 3$ and package stopping defaults; no benchmark target supplied.	ALAMO/BARON local models; common evaluated starts, plus configured sampling.
FMQA-SA	Budget or 1,800 s method limit; no benchmark target supplied.	Rank-eight FM and classical annealing; separate seeded model, annealing, and fallback streams.
Uniform	Budget; no benchmark target supplied.	Uniform unseen integer configurations after the shared starts.
BAM-prefix control	Budget; Grid only.	Preserve BAM callbacks 4–13, then sample uniformly without replacement.

Original process jobs and the additional process comparisons have 2,100 s worker watchdogs; the additional Grid comparisons have the watchdog disabled. Grid BAM trace watchdogs were also disabled, while Grid FMQA-SA used 2,100 s. Seeds are 0–9. Actual repeated/unsuccessful callbacks consume budget; internal cache accesses without callbacks do not. Process DSDA fails if all initial points are unaccepted; starts are never replaced. Surrogate solver calls contribute to search time but are distinct from GAMS/IPOPT evaluations of the original process subproblems. Exact recorded dictionaries and remaining environment limitations are documented in Supplementary Sections S4–S4.1.

3.3.2 Budget-limited LD-SDA-based search (DSDA)

DSDA is the budget-limited implementation used in this study, adapting the neighborhood search and discrete line search of the published LD-SDA method to the external integer lattice (Ovalle et al., 2025). It evaluates the unit L_∞ neighborhood in fixed enumeration order, selects the best improving neighbor, and continues along the resulting direction through discrete line search. An improvement requires an objective decrease greater than 10^{-12}, with ties resolved by the first enumerated neighbor. Neighborhood exploration resumes from the last improving point. If the evaluation budget B is exhausted, the best solution found is retained.

For the process benchmarks, DSDA starts from the best feasible configuration among the three evaluated initial points. For Grid-GDP benchmarks, initial configurations are ranked by objective value, and successive descents are attempted until the evaluation budget is exhausted or the stored target is reached within an absolute tolerance of 10^{-12}. This algorithmic stopping tolerance is distinct from the 10^{-5} assessment tolerance in Equation (10), which is applied to all methods. For all process benchmarks, DSDA searches within the declared integer bounds without precomputed feasibility filtering, using the callback accounting and unaccepted-response policy stated above.

3.3.3 RBFOpt

RBFOpt uses MSRSM, automatic RBF selection, and the default genetic candidate-search option of version 4.3.0 (Costa and Nannicini, 2018; Nannicini, 2021). The settings are `rbf=auto` and `global_search_method=genetic`, with the recorded defaults `ga_base_population_size=400` and `ga_num_generations=20`. The implementation determines the effective population size from the search dimension (Supplementary Section S4). All external variables are integers, and the three shared initial configurations are supplied with additional package initialization disabled. These shared-start and budget settings define the experimental protocol; the implementation is not described as an entirely default package invocation. Other surrogate-minimization operations use Bonmin/IPOPT. These calls contribute to search time but not the objective-evaluation budget. Unaccepted oracle responses follow the common policy stated above. The RBFOpt (solver) setting, results, and paired comparisons are reported in Supplementary Section S10.

3.3.4 BAM

BAM uses local algebraic surrogate models optimized by BARON to generate candidate configurations (Ma et al., 2023; The Optimization Firm, 2026). All external variables are specified as integers using `integrality=1`. The three shared initial configurations are evaluated and supplied through `x_evaluated` and `f_evaluated`, with no additional starting point (`x0=None`). The remaining evaluation budget is set to `max_evals`$=B-3$, while other algorithmic settings retain their defaults. Unaccepted evaluations in the process benchmarks return `NaN`. For the Grid benchmarks, BAM's configured sampling adds a ten-point predetermined Sobol prefix after the three shared evaluations. The first two prefix points account for all additional target hits at a total budget of ten evaluations. Objective evaluations, including repeated calls, are counted independently by the external ledger. For the process experiments, callback B returns its value normally and BAM terminates through its native evaluation limit. All 140 BAM process runs use this stopping procedure with the paired seeds and frozen IPOPT tables.

3.3.5 FMQA-SA

FMQA-SA reuses the PyTorch-based FM training and binary quadratic model (BQM) construction code from the `QHD_Data` branch of `SECQUOIA/bbo_via_fmqa` (SECQUOIA, n.d.). A separate runner in `Black-Box-for-GDP` implements the common DFO comparison protocol and controls the initial points, evaluation budget, response normalization, candidate selection, failure handling, and trajectory logging. At each adaptive iteration, a second-order rank-eight FM with identity activation is initialized with linear and latent weights drawn from a normal distribution with mean zero and standard deviation 0.01, and zero bias. The model is trained on all previously evaluated configurations for 1,000 full-batch epochs using Adam, mean squared error, and a learning rate of 0.01 on CPU in `float32`. Observed finite objective values are min–max scaled

to $[0, 1]$ (or zero when all finite values are identical), while non-finite evaluations receive a training target of 2.0. For reporting and incumbent tracking, non-finite raw objective values are represented by 10^6.

Each external integer coordinate ξ_j is encoded in unsigned binary using $\lfloor \log_2 u_j \rfloor + 1$ bits, where $u_j > 0$ is its upper bound. The fitted FM is converted to a QUBO, with its coefficients scaled by the maximum absolute entry of the quadratic matrix. The archived training code adds no explicit regularization term and does not override Adam's zero weight-decay default. The archived converter has no explicit guard for a zero maximum coefficient, and non-finite BQM coefficients are rejected by the wrapper; no frequency of this edge case is inferred from the retained proposal logs. Binary candidates are generated locally on CPU using D-Wave's classical `SimulatedAnnealingSampler` from `dwave-samplers` 1.6.0, with 100 reads, 1,000 sweeps, and a geometric inverse-temperature schedule (D-Wave Systems, n.d.). The lowest-energy decoded candidate within coordinate bounds that has not previously been evaluated is selected; if none is available, a candidate is drawn uniformly from the remaining unevaluated configurations. GDP logical constraints and NLP feasibility are not screened before oracle evaluation, so infeasible configurations may still consume the evaluation budget. The new observation is appended, and FM fitting and candidate generation repeat until the evaluation budget B or the 1,800 s method time limit is reached. The recorded proposal logs identify uniform unseen-point fallback in 9,522 of 51,450 Grid proposals (18.51%) and 1,573 of 3,010 process proposals (52.26%). Per-run counts and the three-stream random-seed policy are provided in Supplementary Section S4.2 (Table S2 and `fmqa_proposal_counts.csv`).

3.3.6 Sampling controls

Uniform sampling evaluates the same three initial configurations and then draws uniformly without replacement from the remaining declared integer domain. It uses a separate deterministic random stream, no objective-dependent selection, and no logical or NLP-feasibility screening before evaluation. The Grid BAM-prefix control preserves the recorded deterministic callback sequence 4–13 after the three shared starts, truncated at the budget, then continues uniformly without replacement. The preserved prefix was checked across all 1,050 BAM Grid runs; it includes the corner and center evaluations responsible for the early target hits. This control tests the observed fixed sampling design and does not reconstruct proprietary BAM internals. Every callback consumes the same budget and saved point cost under the common acceptance and target rules.

3.4 Execution environment and algebraic comparisons

All computational experiments were conducted on the Dantzig server, equipped with an AMD EPYC 7643 processor (48 cores, 2.3 GHz) and 1 TB RAM. All optimization methods and numerical libraries were configured for single-threaded execution. The software environment included Pyomo 6.10.0, GAMS 52.2, RBFOpt 4.3.0, BAM 5.0.0, PyTorch 2.8.0, and HiGHS 1.12.0. The additional comparisons used the same recorded host, Python/package versions and Bonmin/IPOPT binaries as their historical references. They used CPU 2 rather than historical Grid RBFOpt CPU 0, and the Linux kernel differed; timing differences are observed retained-run comparisons rather than simultaneous controlled microbenchmarks. The Dantzig auxiliary-solver banners identify Bonmin 1.8.9 with Cbc 2.10.12 and Ipopt 3.14.19, and a standalone Ipopt 3.14.19 executable; both are Linux x86_64 builds with ASL 20231111. These executables support surrogate optimization and are separate from the GAMS/IPOPT process interface. BAM uses engine 2026.9.10, build LNX-int-64 dated 2026-09-10 17:59:06. BARON 25.11.17 refers to the GAMS-based algebraic comparisons; the individual ALAMO and BARON versions inside BAM are not reported by its runtime information and remain unidentified.

DFO benchmarking. DSDA, RBFOpt, BAM, FMQA-SA, and the sampling controls use lookup responses under the shared initialization and evaluation-budget protocols described above. Configuration-evaluation costs were measured separately from the searches and combined with recorded method times as described in Section 3.6.

Fixed-disjunction NLP evaluations. We fixed the original GDP Boolean decisions and evaluated the resulting subproblems using GAMS/IPOPT with MUMPS. For the Grid-GDP benchmarks, all 20,181 con-

figurations were evaluated individually, while the process benchmarks used fixed-configuration evaluations of the corresponding process models. IPOPT used a tolerance of 10^{-8} and a 60 s solver time limit per configuration. Process acceptance requires a finite objective, maximum absolute constraint and variable-bound violation of at most 10^{-5}, and termination as `optimal`, `locallyOptimal`, or `globallyOptimal`. The solver status must not be `error` or `aborted`; GAMS must report `SOLVESTAT`=1 and `MODELSTAT`$\in \{1, 2\}$. Time-limited candidates remain unaccepted even when feasible. Supplementary Section S4.1 specifies the residual checks and initialization artifacts. Unaccepted responses do not establish mathematical infeasibility. CSTR sizes 5–25 use the mass-balance/zero-recycle initialization, whereas CSTR30 retains constructor defaults. Column and Small Batch use their saved initialization files, and Dynamic uses fresh clones of the unsolved discretized template.

Algebraic GDP comparisons. The conventional comparisons include GDPopt RIC, GLOA, and LOA, as well as Big-M and Hull reformulations. GDPopt uses HiGHS for MILP master problems and BARON for nonlinear subproblems (Chen et al., 2022; Pyomo Development Team, n.d.). The Big-M and Hull reformulations are solved using BARON as the MINLP solver. All algebraic methods use the original GDP domains, a time limit of 1,800 s per instance, and a relative optimality gap of 10^{-5}.

3.5 Target attainment and cumulative success profiles

Let f_{best} denote the best feasible lookup objective value observed so far and f_{ref} the reference objective value. For process DFO runs, f_{best} is the minimum accepted objective recorded by the experimental wrapper over the charged evaluations considered. A run reaches the reference target when

$$|f_{\text{best}} - f_{\text{ref}}| \leq 10^{-5} \max\{1, |f_{\text{ref}}|\}. \tag{10}$$

Target attainment measures agreement with the reference value rather than certifying global optimality. For the process experiments, f_{ref} is the minimum accepted objective in the frozen IPOPT table, and execution failures count as non-successes even if a target was observed earlier. For process runs, the reported solution is the accepted configuration with the lowest objective recorded by the wrapper within budget. Its coordinates and objective are preserved in the final `runs.csv` summary, including when first observed at callback B. This solution need not equal the native optimizer's returned incumbent. Terminal accepted observations are retained under the same rule in the RBFOpt and uniform comparisons. Supplementary Section S4.3 preserves the historical terminal-callback audit with explicit variant labels, and Section S10 links the additional terminal records. For the full-process-GDP comparison, a feasible objective at or below $f_{\text{ref}} + 10^{-5} \max\{1, |f_{\text{ref}}|\}$ counts as attainment, so solutions better than the reference are also included.

For each evaluation budget B, let $\mathcal{R}_B$ denote the set of problem–seed instances compared across methods, and let s identify a solution method. For each instance $r \in \mathcal{R}_B$, H_{sBr} equals one if the run reaches the target and zero otherwise. Process indicators use the wrapper-reported best-accepted configuration defined above, with execution failures assigned zero. Let E_{sBr} be the index of the first objective evaluation at which the target is reached, with $E_{sBr} = +\infty$ if the target is not reached. At a threshold of N objective evaluations, where $0 \leq N \leq B$, the absolute evaluation-count profile is

$$C_{sB}^{\text{eval}}(N) = \frac{1}{|\mathcal{R}_B|} \sum_{r \in \mathcal{R}_B} \mathbf{1}\{H_{sBr} = 1,\ E_{sBr} \leq N\}. \tag{11}$$

Here, $\mathbf{1}\{\cdot\}$ equals one when the stated conditions hold and zero otherwise, and $|\mathcal{R}_B|$ is the number of problem–seed instances. Thus, $C_{sB}^{\text{eval}}(N)$ gives the fraction of runs that reach the target within N evaluations. For the Grid-GDP benchmarks, $|\mathcal{R}_B| = 210$ at each budget; for each process benchmark and budget, $|\mathcal{R}_B| = 10$. All runs, including target misses, remain in the denominator.

Let $\widehat{T}_{sBr}$ denote the estimated full-run completion time calculated as described in Section 3.6. For a time threshold $t \geq 0$, measured in seconds, the absolute completion-time profile is

$$C_{sB}^{\text{time}}(t) = \frac{1}{|\mathcal{R}_B|} \sum_{r \in \mathcal{R}_B} \mathbf{1}\{H_{sBr} = 1,\ \widehat{T}_{sBr} \leq t\}. \tag{12}$$

This profile gives the fraction of runs that reach the target and have an estimated full-run completion time no greater than t. A successful run is counted at its full completion estimate, not at the time of its first target attainment.

Experiments at different evaluation budgets are analyzed separately; a larger-budget trajectory need not extend a smaller-budget trajectory. Both profiles use absolute evaluation counts or times rather than solver-relative cost ratios (Dolan and Moré, 2002). The evaluation-count profile is related to data profiles (Moré and Wild, 2009), but uses raw rather than dimension-normalized evaluation counts. Algebraic results are reported separately because their iterations and subsolver calls are not DFO objective evaluations, and only one run is reported for each problem–method pair.

3.6 Runtime accounting and restart estimates

For a recorded lookup-based run with n objective evaluations and measured lookup-call times, the estimated full-run completion time is

$$\widehat{T}_{\text{complete}} = T_{\text{recorded}} - \sum_{q=1}^{n} t_q^{\text{lookup}} + \sum_{q=1}^{n} c(\boldsymbol{\xi}^{(q)}) + t_{\text{prep}}. \tag{13}$$

Here, T_{recorded} is the measured runtime of the lookup-based search, t_q^{lookup} is the measured lookup-processing interval of its qth objective call, and $\boldsymbol{\xi}^{(q)}$ is the external integer configuration evaluated in that call. For the process experiments, this interval covers coordinate processing and in-memory lookup, but excludes callback-file writing; the CSV is loaded before the search clock starts. The point cost $c(\boldsymbol{\xi}^{(q)})$ is measured separately for the same benchmark model, problem size, and configuration, including configuration-specific preparation, preprocessing, solver invocation when required, and validation. The term t_{prep} accounts for one-time model preparation not already included in the point costs. The retained search time includes surrogate fitting, candidate generation, auxiliary optimization, and recording performed within the method clock. For the historical process runners, T_{recorded} is the `dfo_wall_s` interval ending after optimizer termination and status checks, immediately before `result.json` is written. For the additional RBFOpt and sampling runs, it is the recorded search-stage interval, with later incumbent extraction and export recorded separately and excluded. Callback logging within that interval is included. The later `summarize()` pass reconstructs the best accepted configuration from the callback records and writes `runs.csv`; these final result-extraction and summary-writing operations are excluded from the completion estimates. Their cost was not measured separately in the historical runs, and no claim that it is negligible is made. Throughout this paper, full-run completion denotes completion of this recorded search stage with reconstructed evaluation costs, excluding final result extraction and export. Every recorded objective call contributes its point cost, including initialization, repeated visits, and preprocessing rejections; the same stored measurement is used each time a configuration is revisited. The reported completion-cost distributions are conditional on this frozen point-cost inventory. They reflect variation in search paths and recorded search overhead, but do not estimate repeated-measurement uncertainty in configuration-evaluation costs.

Grid-GDP timers. Grid-GDP enumeration and objective normalization are excluded from the timing analysis, and $t_{\text{prep}} = 0$ because configuration-specific preparation is included in the point costs. For DSDA on the Grid-GDP benchmarks, lookup-call times were not recorded, so the method runtime is retained without subtracting lookup time.

Process timers. Each original recorded point cost is preserved with its objective and status. Separately measured rejection costs cover the structural exclusions: 435 for CSTR30, 17 for Column, and 35 for Dynamic. These coordinates follow the same model preparation and logical-rejection paths, with no NLP solver invocation. Their positive total processing times are measured rather than set to zero. Costs are joined by the complete model/size/coordinate key, never by the coordinate tuple alone. No cross-size CSTR timing substitution or successful-retry selection is used. For Dynamic, the original model-template construction and discretization cost is included once per run through t_{prep}; additional campaign-level preparation is not charged repeatedly. All in-budget evaluations have measured point costs, and estimated full-run completion times are available for all 700 primary process runs. The sole execution failure, a Column DSDA initialization

failure, retains its recorded cost and contributes zero target success. These estimates retain the recorded lookup trajectories rather than measuring online optimizer–NLP executions.

Algebraic GDP completion times are measured separately and are not augmented with the DFO point costs. Grid-GDP algebraic timings include per-run preparation, reformulation, solution, and validation, but exclude shared-template preparation and lattice enumeration. Full-process-GDP timings include model-template preparation.

Independent-restart estimate. For each problem, method, and evaluation budget, let $n_{\mathrm{rep}} = 10$ denote the number of runs and h the number reaching the reference target. The empirical success fraction is $\widehat{p} = h/n_{\mathrm{rep}}$. For process runs, h counts wrapper-reported best-accepted configurations satisfying the target criterion in Section 3.5, including accepted terminal configurations preserved in the final run summaries. Thus the process restart estimate concerns this configuration-discovery event, not return of a target-attaining native optimizer incumbent. Let $\widetilde{T}$ denote the median estimated full-run completion time across all n_{rep} runs, including target misses. For a per-run success probability p, let $R_{99}(p)$ denote the total number of independent runs required to achieve at least a 99% probability of reaching the target at least once. The empirical median-cost restart estimate, denoted $\widehat{\mathrm{TTS}}_{99}$, is

$$\widehat{\mathrm{TTS}}_{99} = \widetilde{T}\, R_{99}(\widehat{p}), \qquad R_{99}(p) = \begin{cases} \lceil \ln(0.01)/\ln(1-p) \rceil, & 0 < p < 1, \\ 1, & p = 1, \\ \infty, & p = 0. \end{cases} \tag{14}$$

Here, $\lceil \cdot \rceil$ denotes rounding up to the nearest integer. The calculation assumes independent restarts, substitutes the observed success fraction for the unknown success probability, and uses $\widetilde{T}$ as the representative cost of each run. With ten observations, it is a descriptive restart estimate rather than a statistical guarantee of 99% success or a 99th percentile of solution time. Supplementary Section S7, Tables S8–S13, reports pointwise 95% exact binomial intervals for every problem–method–budget success fraction: ten hits yield $[0.6915, 1]$, while zero hits yield $[0, 0.3085]$. These intervals assume independent seed-level trials under the fixed protocol and are not simultaneous intervals. The plug-in endpoints and restart formula are retained. Hit counts and completion-cost distributions are the primary evidence; TTS99 is a secondary summary. Grid uncertainty is assessed within each function, preserving the problem–seed pairing, rather than pooling heterogeneous functions as one binomial sample. The median TTS99 across the 21 Grid-GDP benchmark functions includes infinite estimates. TTS99 is not calculated for the algebraic methods, which have only one run per problem–method pair.

4 Computational Results and Discussion

4.1 Search Performance on Analytical Grid-GDP Benchmarks

The evaluation budget changes the observed ordering of the search implementations and sampling controls (Table 3). At ten evaluations, BAM and its prefix control each reach 111 of 210 targets (52.9%), compared with 42 (20.0%) for RBFOpt and 14 (6.7%) for uniform sampling. Of BAM's 111 successes, 11 occur during shared initialization and 100 at two predetermined Sobol sampling points (10 at a corner and 90 at the grid center), before surrogate-guided candidate selection. At all tested budgets from 30 onward, RBFOpt achieves the highest aggregate target-hit fraction, reaching 171 of 210 runs (81.4%) at 100 evaluations, compared with 147 (70.0%) for BAM, 124 (59.0%) for the prefix control, and 45 (21.4%) for uniform sampling. The prefix control reproduces every BAM B=10 target classification and best objective. At B=100, BAM adds 23 hits over that control, indicating a benefit of its subsequent search on these inputs. Equal BAM and control hit totals at B=30 do not imply identical successful seeds. These observations establish the importance of the fixed design at low budgets without claiming a full causal decomposition of BAM.

Target-set cardinality varies substantially across the analytical benchmarks (Supplementary Table S6). In particular, 870 of 961 configurations (approximately 90.5%) satisfy the Xin-She Yang 3 (historical variant) target, so this instance contributes ten successes per method even at the smallest budget, $B = 10$ (Supplementary Table S9). Aggregate attainment therefore combines search behavior with heterogeneous target-set

difficulty. Excluding both Xin-She Yang 3 and the nearly redundant sampled Bohachevsky 2 case leaves the B=100 ordering unchanged: RBFOpt 79.47%, BAM 66.84%, FMQA-SA 57.89%, prefix control 54.74%, DSDA 44.21%, and uniform 17.89% (Supplementary Section S10).

Table 3: Grid-GDP reference-target hit percentages from the recorded trajectories. Each cell contains 210 runs (21 functions and ten seeds). The evaluation budget includes the three shared initial evaluations, and repeated objective callbacks are counted toward the evaluation budget. BAM's 111 hits at $B = 10$ all precede surrogate-guided selection (11 shared-initialization hits and 100 at two predetermined Sobol points). Per-function integer hit counts are in Supplementary Tables S9–S13.

Budget B	DSDA	RBFOpt	BAM	FMQA-SA	Uniform	BAM-prefix control
10	6.2	20.0	52.9	7.6	6.7	52.9
30	35.7	61.9	54.8	22.4	11.9	54.8
50	42.4	73.3	57.6	36.2	13.8	56.7
70	48.1	76.7	62.9	48.6	17.1	57.1
100	49.5	81.4	70.0	61.4	21.4	59.0

Figure 2 shows the cumulative fraction of runs attaining the reference target as a function of the number of objective evaluations, revealing when successful searches first reach the target. Each panel is a separate budget experiment, so trajectories at a larger evaluation budget are not assumed to extend those at a smaller evaluation budget. DSDA attains more targets than FMQA-SA at evaluation budgets of 30 and 50, whereas their aggregate attainment is nearly identical at an evaluation budget of 70 (101 versus 102 successful runs). At the largest tested evaluation budget of 100, FMQA-SA reaches 129 targets compared with 104 for DSDA.

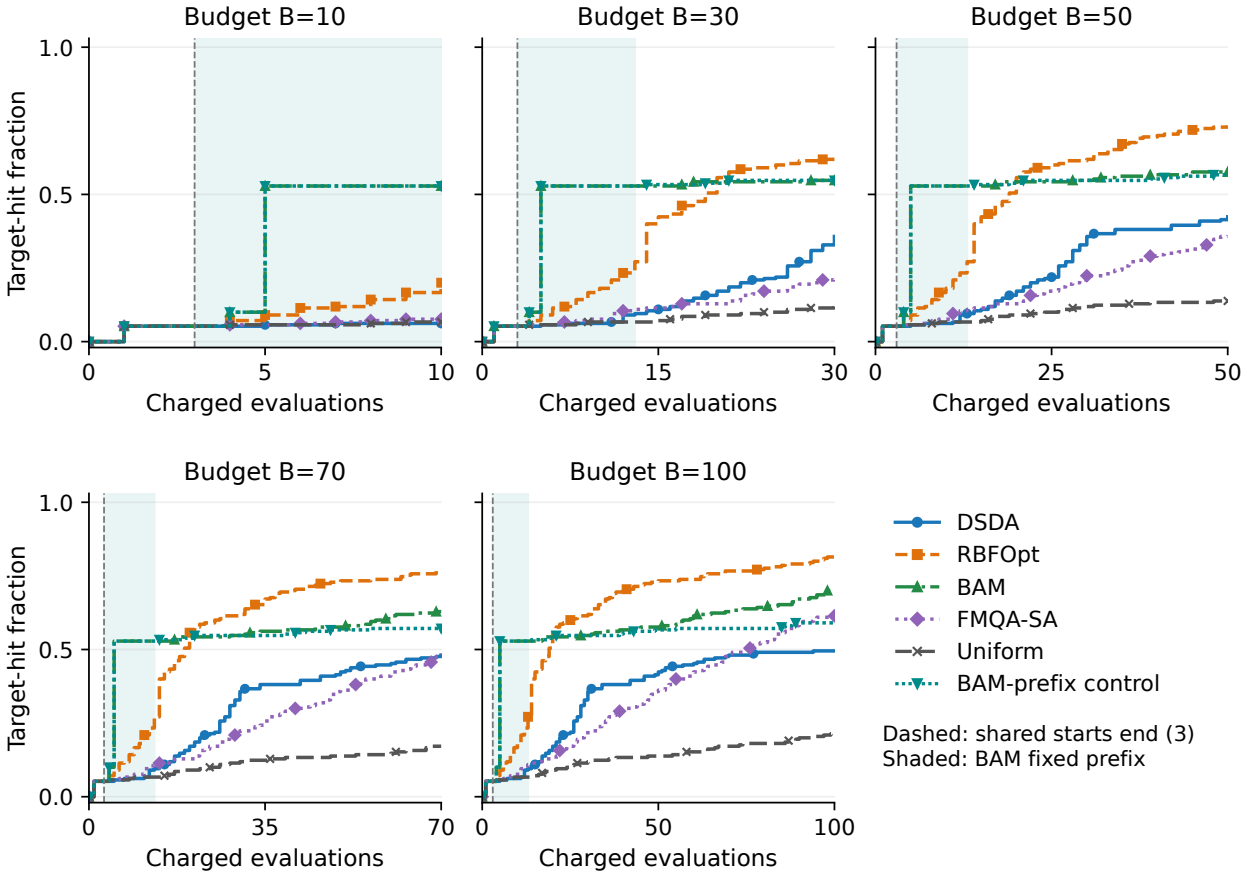


Figure 2: Absolute evaluation-count cumulative success profiles for four search implementations and two sampling controls. The horizontal axis is the number of objective callbacks to first target attainment; the vertical axis is the fraction of all 210 problem–seed instances reaching the target within that count. Initial and repeated callbacks count, unsuccessful runs remain in the denominator, and counts are not normalized by another solver. The panels use evaluation budgets of 10, 30, 50, 70, and 100; colors, line styles, and markers distinguish methods. The gray dashed line at three evaluations marks the end of the shared initialization. Shading from callback 4 through the smaller of 13 and the budget marks the preserved BAM sampling prefix.

4.2 Computational Comparison on Grid-GDP Problems

The analytical formulation in Section 3.1 permits comparison with algebraic methods and a specified fixed-point evaluation protocol. Completion estimates for the 6,300 primary Grid runs attach individually measured GAMS/IPOPT costs for all 20,181 coordinates to 311,033 recorded callbacks. Every actual callback, including a repeated visit, receives the cost of its coordinate under the accounting rules in Section 3.6.

The fixed-point measurements have mean and median costs of 0.6522 and 0.3727 s, respectively; the mean is higher because some external solver calls incur long delays. Because both coordinates are fixed, the measured point costs primarily reflect the fixed-configuration evaluation protocol, including solver-interface and evaluation overhead, rather than the solution of an NLP with free continuous decision variables. At $B = 100$, RBFOpt reaches more targets than BAM with a median completion estimate of 70.81 versus 64.42 s (Table 4). Their median summed point costs are 60.84 and 63.97 s, while their median retained search components are 7.579 and 0.430 s. Uniform and the BAM-prefix control have median costs of 60.51 and 61.81 s, respectively, but lower target coverage. FMQA-SA's 62.04 s median search component is comparable to its 62.34 s median summed point cost, contributing to a 124.13 s median completion estimate. DSDA has a small retained search component (approximately 0.0032 s) and can terminate below the evaluation budget, giving a median completion estimate of 24.14 s alongside its lower target coverage. The medians of separate cost components need not sum to the median total. The primary-variant callback counts and first-hit records are provided in `manuscript/tables/run_records.csv`, helping distinguish early termination from the costs of the configurations evaluated.

Table 4: Grid-GDP DFO results at $B = 100$, with 210 runs per method. All time columns are medians over target hits and misses. Component medians need not sum to the completion median; none of these columns is a first-hit time or TTS99 estimate. The historical stopping audit is preserved in Supplementary Table S4 and `stopping_diagnostics.json`; the primary records are identified in Section S10.

Method	Target hits	Median full-run completion (s)	Median summed point cost (s)	Median retained search time (s)
DSDA	104/210	24.14	24.14	0.0032
RBFOpt	171/210	70.81	60.84	7.5790
BAM	147/210	64.42	63.97	0.4301
FMQA-SA	129/210	124.13	62.34	62.0359
Uniform	45/210	60.51	60.51	0.0026
BAM-prefix control	124/210	61.81	61.80	0.0026

Figure 3 shows how target coverage accumulates with estimated full-run completion time under each tested budget. Figure 4 combines empirical hit rates with median estimated full-run completion times under the independent-restart assumption. At $B = 100$, the medians of the 21 problem-specific TTS99 estimates are 194.10 s for DSDA, 73.80 s for RBFOpt, 84.77 s for BAM, 757.81 s for FMQA-SA, 2,202.89 s for uniform, and 83.22 s for the prefix control, retaining infinite values. DSDA and BAM nevertheless have zero observed hits on four and two functions, respectively; RBFOpt and FMQA-SA have at least one hit on every function at that evaluation budget. Uniform has seven zero-hit functions and the prefix control has two. Suite medians should therefore be read alongside zero-hit counts; for a known problem, its own hit rate and completion cost are more informative. The ten-seed TTS99 values retain the descriptive interpretation and uncertainty limits in Section 3.6.

Table 5 reports the algebraic solver results separately because their iterations and subsolver calls are not equivalent to DFO callbacks and one selected run is included per problem and method. The algebraic solver families and configurations are described in Section 3.4. Each GDPopt method—RIC, GLOA, and LOA—attains the reference target on 18 of 21 functions; all three miss Hölder table, Michalewicz, and Shubert (historical variant). Target attainment can coexist with termination at a time limit, as in all three GDPopt runs for Three-hump camel and the Lévy 13/RIC run. Consequently, the measured completion time of a target-attaining run must be distinguished from a completed global-optimality certification.

Big-M/BARON attains four of 21 targets and Hull/BARON five of 21. Each has 16 unsupported functions because the configured GAMS/BARON interface does not accept the required sine/cosine expressions. These entries are solver-compatibility limitations, not optimization timeouts or mathematical failures of

either reformulation. Among the five supported functions, target-attaining runs have measured worker completion times between approximately 0.55 and 1.30 s; Hull attains all five targets, whereas Big-M misses Styblinski–Tang. The ten executed target misses are marked M, even when the raw solver termination is optimal; unsupported cases are marked U and remain in the denominator. Only the 63 target-attaining runs receive numerical times in the table; times for all ten executed target misses are retained in the complete algebraic-run index linked in Supplementary Section S8. For example, the target-attaining Rosenbrock runs complete in 0.758 s with Big-M and 0.813 s with Hull. These supported cases demonstrate that access to the algebraic structure can be highly competitive, while the full-suite hit fractions additionally reflect implementation coverage.

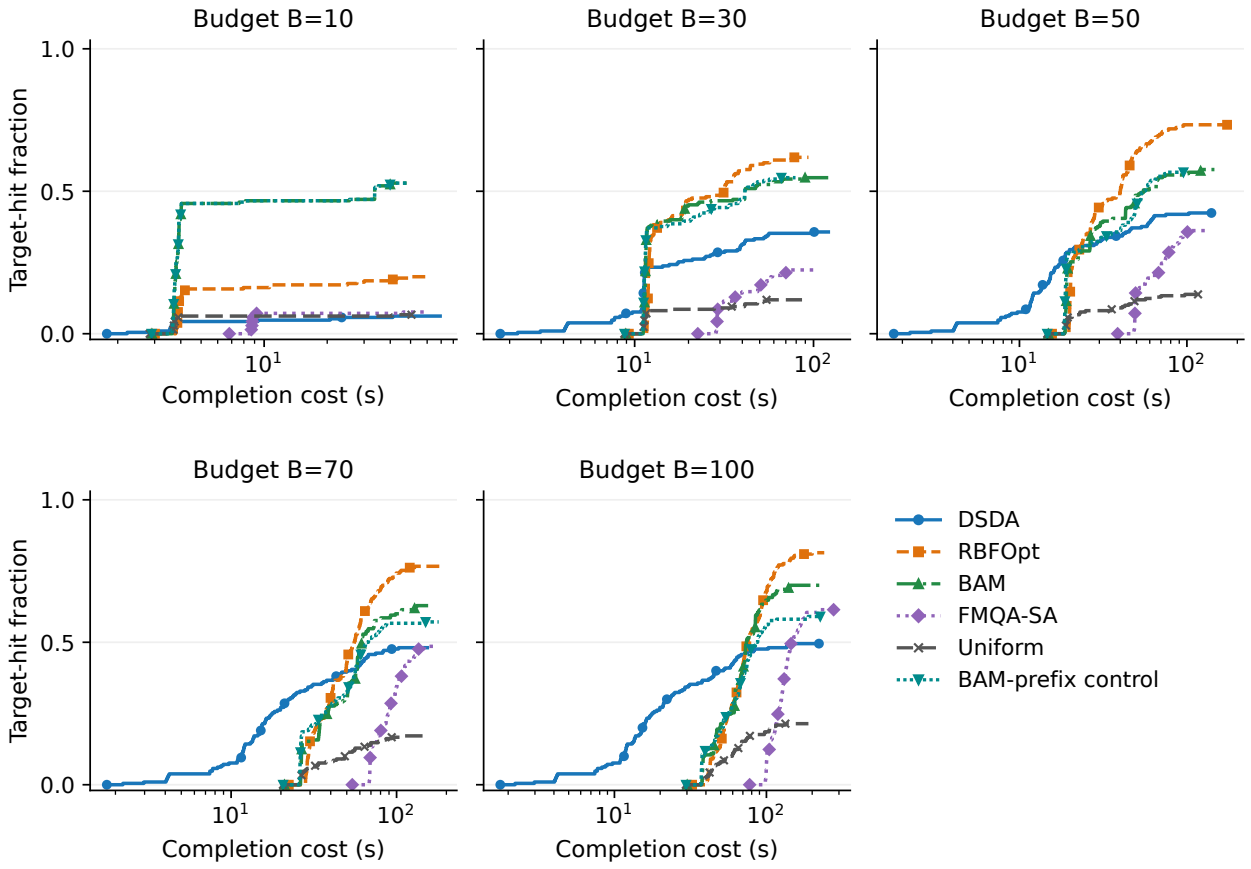


Figure 3: Absolute completion-time cumulative success profiles for Grid-GDP. The horizontal axis is estimated full-run completion time in seconds on a logarithmic scale; the vertical axis is the fraction of all 210 problem–seed instances that reached the target and completed within that time. Estimates combine recorded search overhead with the sum of measured fixed-point GAMS/IPOPT costs along the complete recorded path. Misses retain their finite completion estimates in the run data but contribute no success. These are neither first-hit-time curves nor solver-relative ratios. Configured stopping policies differ: Grid DSDA may stop at the target or after completing local searches, while the selected RBFOpt, BAM, and FMQA-SA runs reach their evaluation budgets. GDPopt and reformulations are excluded. The cost decomposition at $B = 100$ is given in Table 4.

Table 5: Target attainment and measured completion times for the selected Grid-GDP runs. Each cell represents one run for one problem and method. Numerical entries are measured full-worker completion times in seconds for target-attaining runs only; they are not times to first target attainment. Target attainment does not certify global optimality.

Problem	GDPopt RIC	GDPopt GLOA	GDPopt LOA	Big-M/ BARON	Hull/ BARON
Ackley	899.10	909.69	310.47	U	U
Alpine 1	0.88	0.85	0.95^{*}	U	U
Alpine 2	1789.14^{T}	1789.40^{T}	1795.11^{T}	U	U
Bohachevsky 2	175.17	223.44	154.58	U	U
Three-hump camel	$1800.10^{\mathrm{T}*}$	$1789.34^{\mathrm{T}*}$	$1789.64^{\mathrm{T}*}$	1.15	1.00
Csendes (squared variant)	0.92	0.94	1.21	U	U
Deflected corrugated spring	507.80	440.76	378.11	U	U
Drop-Wave	120.15^{*}	176.69^{*}	221.96^{*}	U	U
Easom	1176.78	1769.44	1787.95	U	U
Griewank	1214.84	1581.92	1042.31	U	U
Hölder table	**M**	**M**	**M**	U	U
Hosaki	1569.06	1524.42	1758.69	1.30	1.29
Lévy	1731.14	1746.53	1794.67^{T}	U	U
Lévy 13	1800.10^{T}	1789.92^{T}	1774.92	U	U
Michalewicz	**M**	**M**	**M**	U	U
Rastrigin	78.26	114.81	180.11	U	U
Rosenbrock	334.94	278.13	655.39	0.76	0.81
Shubert (historical variant)	**M**	**M**	**M**	U	U
Styblinski–Tang	1339.67	1440.10	1562.35	**M**	0.95
Sum of squares	822.85^{*}	754.81^{*}	851.15^{*}	0.55	0.58
Xin-She Yang 3 (historical variant)	1.01	0.90	1.37	U	U
Target hits / all problems	18/21	18/21	18/21	4/21	5/21
Target hits / supported problems	18/21	18/21	18/21	4/5	5/5
Executed target misses	3	3	3	1	0
Unsupported	0	0	0	16	16

M: an executed run that did not attain the reference target, including runs whose raw solver termination is optimal. U: unsupported expression. Overall attainment uses all 21 problems; supported-case attainment excludes U entries and measures performance conditional on interface compatibility.

T: termination at a time limit. A target-attaining run retains its numerical time and T marker. *: runs from the `pyomo610-grid-v1` execution profile; these ten selected runs are distinct from the ten target misses marked M.

Worker times include worker-side preparation, transformation, solve, and validation, and exclude shared-template preparation and lattice enumeration. The ten starred entries replace previously failing cases using the explicitly archived `pyomo610-grid-v1` correction profile, without best-time or best-objective selection. The analytic expressions, reference CSVs, solver settings and 1,800 s budgets are unchanged; the profile corrects fixed-exponent FBBT propagation and the Alpine1/LOA convergence check before another cut. The source records retain times for executed target misses, raw statuses, objectives and hashes; Supplementary Section S8 links the full index and archives.

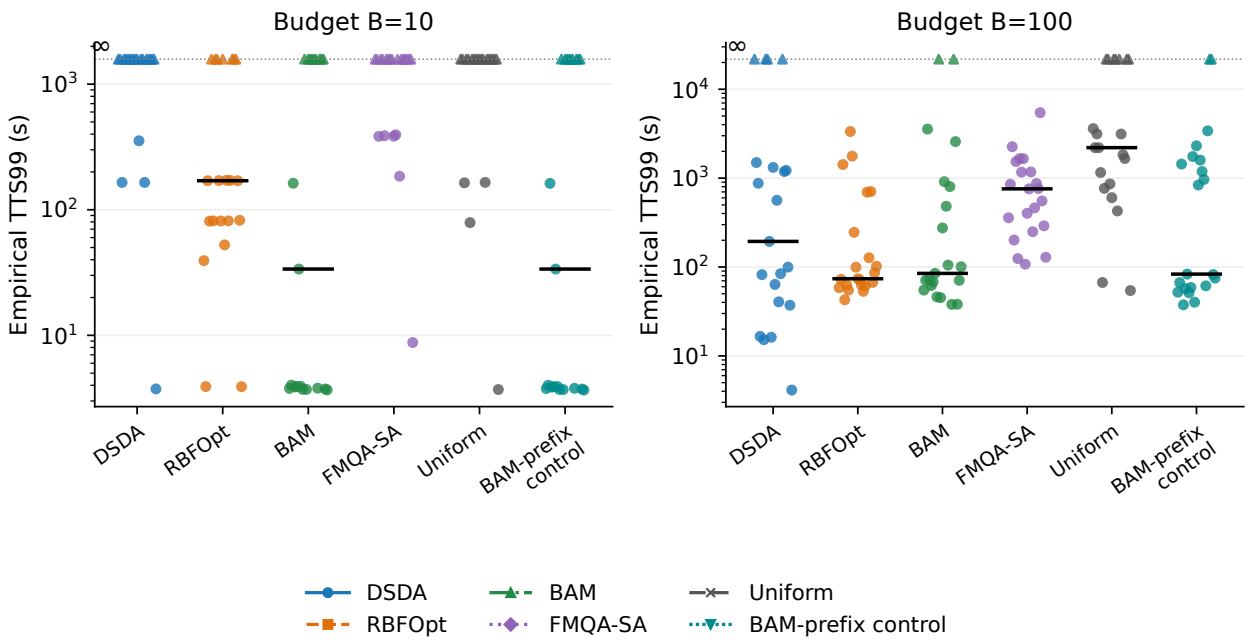


Figure 4: Empirical TTS99 for each of the 21 Grid functions at budgets 10 and 100. Each dot is a function-specific ten-seed estimate; black bars denote suite medians when finite. Triangles on the upper dotted line denote infinite estimates from zero observed hits and are not assigned a finite runtime. The vertical axis is logarithmic. These estimates use complete-run median costs and are neither first-target times nor 99% guarantees.

4.3 Process GDP Case Studies

The process benchmarks expose differences in target attainment when configuration evaluations can be accepted, rejected during preprocessing, or unsuccessful under the fixed local-solver policy.

4.3.1 Target attainment on the recorded lookup paths

The process experiments use IPOPT-generated objective tables and ten paired seeds per condition. All evaluations within the declared bounds, including structural rejections, failed evaluations, and repeated calls, count toward the evaluation budget. Across the 700 primary runs, 326 reach the target, 373 miss it, and one fails during execution; the failure remains in the denominator (Table 6).

RBFOpt reaches the CSTR target in 56 of 60 runs across the six size-dependent budgets, compared with 15 for DSDA, 18 for BAM, nine for FMQA-SA, and 11 for uniform sampling. At CSTR30 and $B = 43$, RBFOpt reaches all ten targets, DSDA three, BAM two, and FMQA-SA one. These results favor RBFOpt under the tested budgets; model size and budget increase together, and NLP initialization changes at CSTR30.

For Column, DSDA and RBFOpt reach five targets each, FMQA-SA four, and BAM and uniform none. All methods use the restricted 81-configuration box. The failed DSDA run with three unaccepted initial points is retained rather than replaced by a different seed. Small Batch uses all 27 configurations. At $B = 10$, DSDA reaches eight targets, FMQA-SA six, RBFOpt five, and BAM two.

For Dynamic, RBFOpt reaches all ten targets at every tested budget, beginning at $B = 12$. DSDA reaches all ten from $B = 30$ onward. BAM reaches four targets at $B = 30$, seven at $B = 37$, and all ten at $B = 50$. FMQA-SA reaches six targets at $B = 50$. All 140 BAM runs terminate at their declared evaluation limits, with complete records for all 3,430 evaluations and no budget-control failures.

The process fallback rate of 52.26% reported in Section 3.3.5 comprises pooled rates of 27.41% for CSTR, 63.08% for Column, 72.81% for Dynamic, and 82.86% for Small Batch (Supplementary Section S4.2). FMQA-SA reaches 42/140 process targets compared with 36/140 for uniform, whereas its Grid B=100 count is 129/210 compared with 45/210. The observed benefit over sampling differs across the two suites and

concerns the complete implementation, without isolating fitting, annealing, or fallback contributions. Seven reconstructed FM fits support limited conversion and proposal-eligibility checks, but do not verify the unsaved historical weights or QUBOs (Supplementary Section S10).

Table 6: Reference-target attainment in the IPOPT-table process experiments. Entries are target hits of the wrapper-reported best-accepted configuration divided by all ten paired seeds at evaluation budget B.

Case	B	DSDA	RBFOpt	BAM	FMQA-SA	Uniform
Column	16	$5/10^{F1}$	5/10	0/10	4/10	0/10
CSTR 5	8	2/10	10/10	10/10	3/10	4/10
CSTR 10	15	2/10	9/10	1/10	2/10	1/10
CSTR 15	22	2/10	8/10	1/10	1/10	2/10
CSTR 20	29	3/10	9/10	2/10	1/10	2/10
CSTR 25	36	3/10	10/10	2/10	1/10	1/10
CSTR 30	43	3/10	10/10	2/10	1/10	1/10
Small Batch	10	8/10	5/10	2/10	6/10	4/10
Dynamic	12	4/10	10/10	0/10	2/10	0/10
Dynamic	15	5/10	10/10	3/10	3/10	3/10
Dynamic	20	8/10	10/10	3/10	3/10	3/10
Dynamic	30	10/10	10/10	4/10	4/10	4/10
Dynamic	37	10/10	10/10	7/10	5/10	5/10
Dynamic	50	10/10	10/10	10/10	6/10	6/10

Superscript Fn denotes n execution failures, which remain in the denominator and do not count as hits. The only failure is a Column DSDA initialization failure. All 70 primary groups use seeds 0–9. All methods use the stated initialization and stopping policies. An accepted target configuration at callback B counts because its coordinates and objective are preserved in the final wrapper summary. BAM also returns that value to its native optimizer; the other methods retain the wrapper stop (Supplementary Section S4.3, Table S3). The reference is the accepted IPOPT-table minimum, not a global-optimality certificate.

Figure 5 shows the first target-attaining evaluation on successful recorded runs, with misses and execution failures retained in the denominator.

4.3.2 Evaluation costs and objective consistency

Table 1 and the timing analysis use one final inventory of 2,455 case/size/coordinate keys across the nine models. The primary process comparison contains 16,252 charged callbacks. Every callback, including unsuccessful and repeated evaluations, receives its saved cost using the complete case/size/configuration key. The additional RBFOpt and uniform ledgers were independently checked against the frozen objective, acceptance, penalty and point-cost records. No original process NLP was rerun for these comparisons; failures to obtain accepted solutions do not establish mathematical infeasibility. Estimated full-run completion times are available for every seed.

The costs combine recorded NLP evaluations with separately measured structural-rejection processing. Detailed reconciliation and source manifests are provided in Supplementary Sections S4 and S8.

The audit establishes consistency of the retrospective cost reconstruction, not independent online replication or global optimality. Conclusions about search performance are conditional on this fixed local-solver oracle.

Figure 6 separates optimizer/search overhead from point-evaluation and one-time preparation costs for four illustrative conditions. For RBFOpt, the search share is 2.40% for Column, 3.12% for Small Batch, 8.26% for CSTR30, and 42.67% for Dynamic at $B = 50$. DSDA's search share is below 0.2% in these four conditions. The retained search-time component includes all operations retained within the method clock after subtracting measured lookup-processing time, rather than isolated surrogate-solving time. Dynamic DSDA executes 221 callbacks across ten $B = 50$ runs, compared with 500 for each of RBFOpt, BAM, and FMQA-SA. Its lower cost reflects shorter paths as well as low search overhead; unsuccessful candidates are evaluated and charged within the common box.

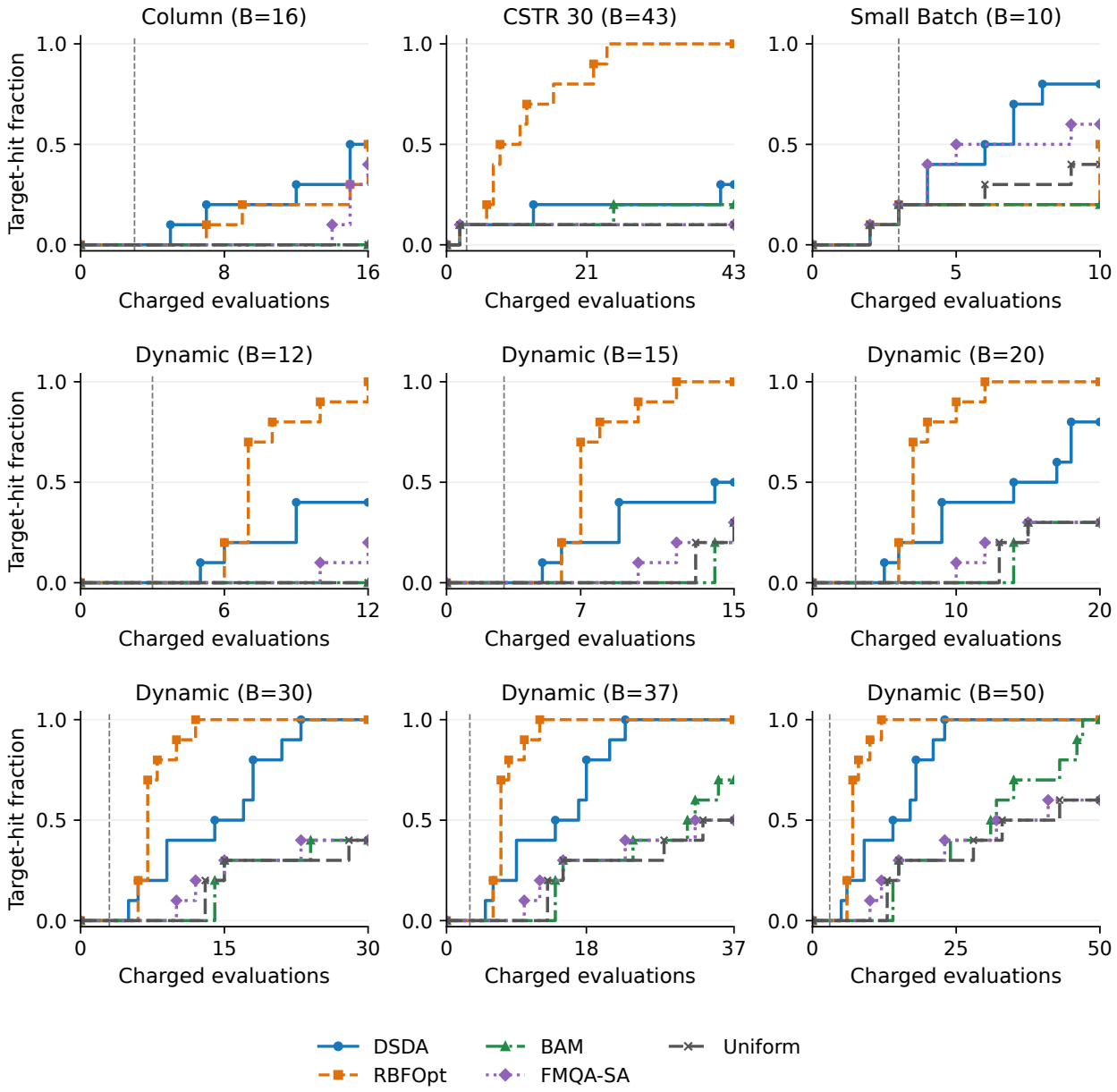


Figure 5: Cumulative target attainment versus charged evaluations for nine illustrative process conditions, using the wrapper-reported success definition in Section 3.5. Every method–condition group retains all ten seeds, including execution failures. Initialization, unsuccessful evaluations, and repeated callbacks consume the budget. Steps mark the first accepted target observation on successful runs. The gray vertical line at three evaluations marks the end of the shared initialization. Line styles and markers distinguish methods. All 14 conditions appear in Table 6.

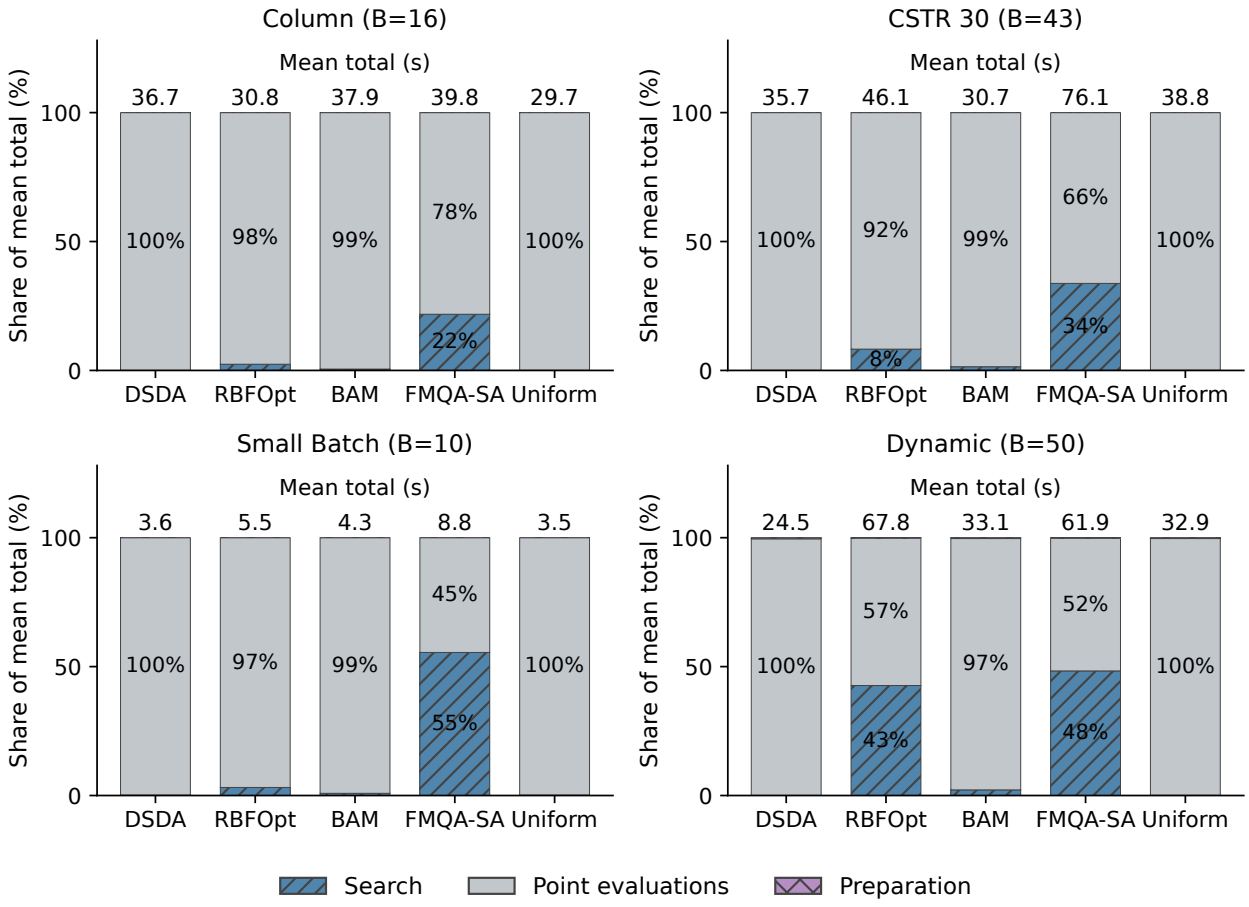


Figure 6: Relative computational costs for four process conditions. Each bar uses all ten seeds, including target misses and the timed execution failure; shares are $100\overline{T}_k/\overline{T}_{\text{total}}$, and the numbers labeled 'Mean total (s)' above the bars give mean reconstructed completion times. Hatching denotes search overhead, gray denotes point evaluations, and crosshatching denotes one-time Dynamic preparation. Shares below 8% are unlabeled. Every in-budget repeated or unsuccessful evaluation is charged. Structural-rejection costs are measured separately; each NLP point cost is taken from the same recorded attempt as its objective and acceptance status.

4.3.3 Absolute completion profiles and restart estimates

Figure 7 presents target attainment against estimated full-run completion time for the nine illustrative conditions in Figure 5. For Dynamic at $B = 30$, DSDA and RBFOpt both reach ten targets, with median completion estimates of 26.61 and 34.19 s, respectively. RBFOpt already reaches all ten at $B = 12$ with a median of 12.29 s. Thus the comparison depends on whether the evaluation budget is fixed or selected separately for each method.

Table 7 reports median estimated full-run completion times over all ten runs and empirical TTS99 values for all 14 conditions. Of the 70 groups, 66 have finite TTS99 estimates. BAM and uniform each have zero hits for Column at $B = 16$ and Dynamic at $B = 12$; these four estimates are infinite, and no group has missing timing data. For CSTR30, RBFOpt reaches all ten targets and has a TTS99 of 38.89 s, compared with 506.21 s for DSDA (three hits) and 621.30 s for BAM (two hits). Their median completion costs are 38.89, 38.94, and 29.59 s, respectively. For Dynamic at $B = 50$, DSDA, RBFOpt, and BAM each reach the target in all ten runs. BAM's TTS99 is 33.98 s, between DSDA's 26.60 s and RBFOpt's 65.14 s. For Small Batch, DSDA has the lowest TTS99 at 11.50 s, compared with 34.87 s for uniform and 35.84 s for RBFOpt. These estimates combine empirical success fractions with complete-run medians and do not provide statistical 99% guarantees.

Figure 8 shows the variation among individual timed runs, distinguishing hits, misses, and execution failures.

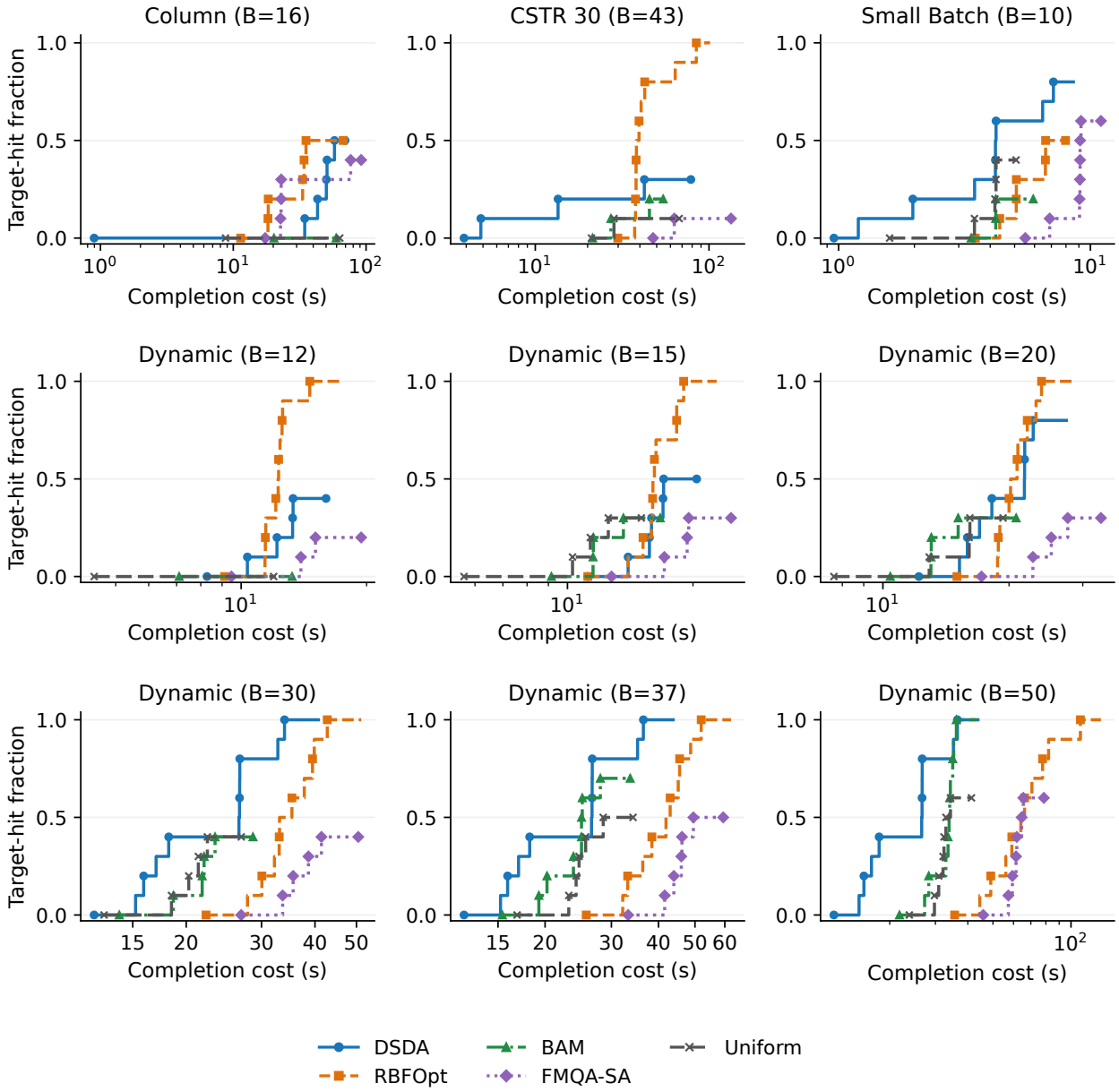


Figure 7: Absolute cumulative target-attainment profiles for nine process conditions. Reconstructed search-completion costs exclude the final wrapper result-extraction and summary-writing pass; see Section 3.6. The horizontal axis is reconstructed full-run completion time on a logarithmic scale; all ten seeds remain in each denominator. Only runs whose wrapper-reported best-accepted configuration attains the target contribute steps. Target misses and execution failures contribute no success. Every run has a complete cost estimate. These are fixed-path estimates, not first-hit-time curves or solver-relative ratios. All 14 conditions are summarized in Table 7.

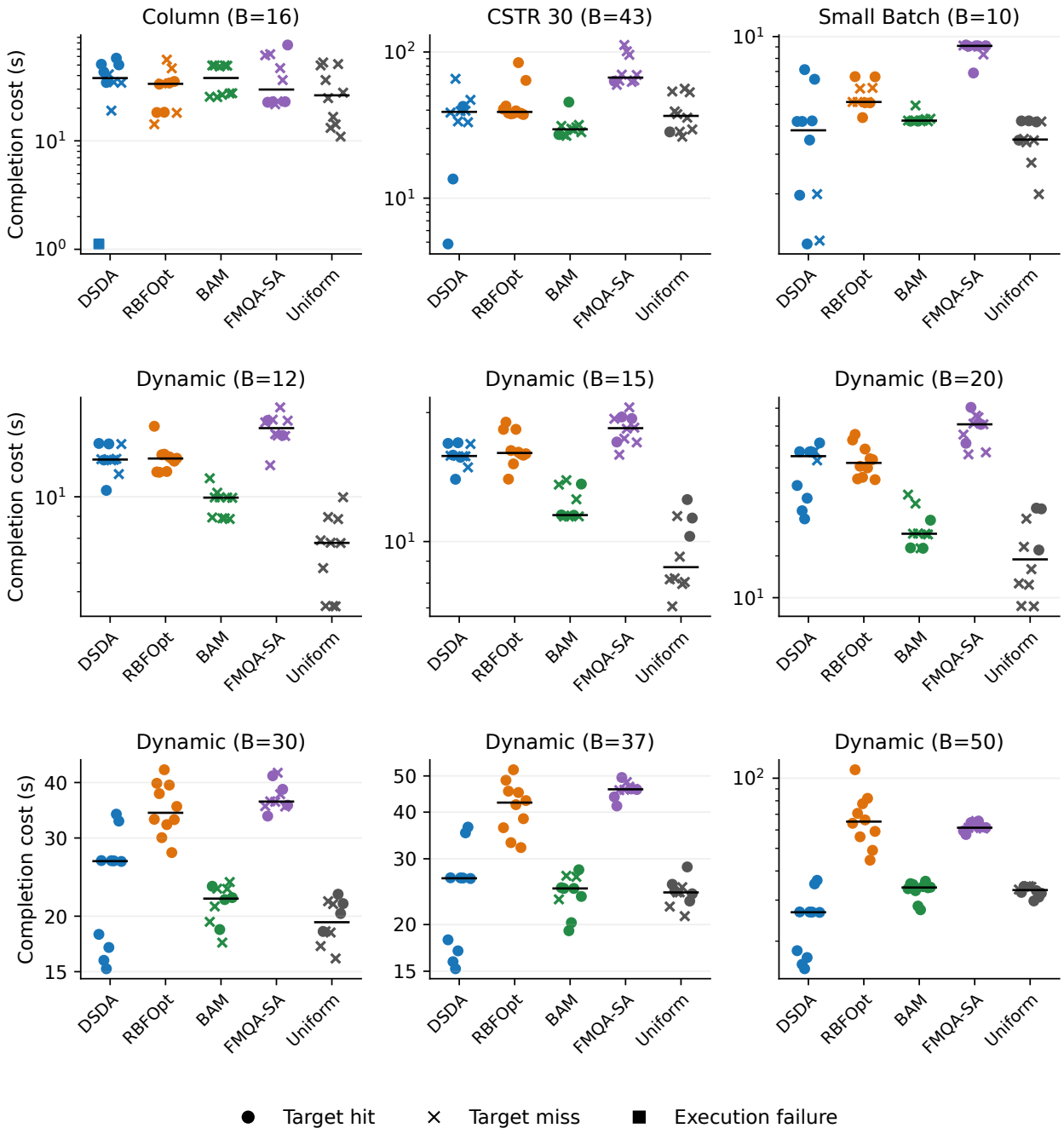


Figure 8: Individual reconstructed completion times for nine process conditions. Reconstructed search-completion costs exclude the final wrapper result-extraction and summary-writing pass; see Section 3.6. Circles denote target hits, crosses target misses, and filled squares timed execution failures; horizontal offsets follow seed order. Black lines are medians over all ten seeds, whose completion costs are available in every group. Misses retain finite costs. The spread is conditional on the frozen point-cost inventory and reflects search-path and recorded-overhead variation, not repeated NLP timing uncertainty. These points are neither first-target times nor TTS99 estimates.

Table 7: Process full-run completion costs and empirical median-cost restart estimates, in seconds. Both panels use all ten paired seeds, including misses and timed execution failures.

(a) Median reconstructed full-run completion cost

Case	B	DSDA	RBFOpt	BAM	FMQA-SA	Uniform
Column	16	37.91	33.54	37.97	29.72	26.29
CSTR 5	8	10.66	10.45	10.67	13.92	9.44
CSTR 10	15	20.98	20.09	18.54	27.07	17.75
CSTR 15	22	29.92	31.85	27.67	42.82	27.84
CSTR 20	29	40.67	44.47	39.37	59.10	40.12
CSTR 25	36	51.67	56.86	50.47	74.70	51.17
CSTR 30	43	38.94	38.89	29.59	66.79	36.54
Small Batch	10	3.83	5.12	4.23	9.09	3.49
Dynamic	12	12.22	12.29	9.95	14.47	7.80
Dynamic	15	15.84	16.10	11.53	18.39	8.71
Dynamic	20	21.29	20.54	14.07	25.23	12.27
Dynamic	30	26.61	34.19	21.91	36.24	19.38
Dynamic	37	26.60	42.42	24.99	46.05	24.38
Dynamic	50	26.60	65.14	33.98	61.31	33.12

(b) Empirical median-cost restart estimate (TTS99)

Case	B	DSDA	RBFOpt	BAM	FMQA-SA	Uniform
Column	16	265.36	234.79	∞	297.25	∞
CSTR 5	8	223.89	10.45	10.67	180.98	94.38
CSTR 10	15	440.66	40.17	815.87	568.40	781.18
CSTR 15	22	628.31	95.55	1217.37	1883.92	584.67
CSTR 20	29	528.71	88.94	826.80	2600.34	842.45
CSTR 25	36	671.73	56.86	1059.97	3286.67	2251.32
CSTR 30	43	506.21	38.89	621.30	2938.80	1607.64
Small Batch	10	11.50	35.84	88.88	54.57	34.87
Dynamic	12	122.17	12.29	∞	303.79	∞
Dynamic	15	110.87	16.10	149.85	239.11	113.28
Dynamic	20	63.87	20.54	182.90	328.03	159.51
Dynamic	30	26.61	34.19	219.08	362.38	193.85
Dynamic	37	26.60	42.42	99.95	322.34	170.67
Dynamic	50	26.60	65.14	33.98	367.89	198.72

TTS99 follows Equation (14), using unrounded ten-seed medians; infinity denotes zero observed hits. Panel (a) reports completion costs, not first-hit times. Reconstructed search-completion costs exclude the final wrapper result-extraction and summary-writing pass; see Section 3.6. Panel (b) is a descriptive plug-in restart estimate, not a 99% guarantee or a runtime quantile. Pointwise success-rate intervals are supplied in Supplementary Table S8 (Section S7). Every group has complete measured point-cost coverage, with no zero filling or cross-size substitution.

Table 8 compares five algebraic methods on nine process GDP models. All 45 saved solutions are numerically feasible under the original-GDP residual test with threshold 10^{-5}. The target-attainment counts are 3/9 for RIC, 1/9 for GLOA, 4/9 for LOA, 7/9 for Big-M, and 6/9 for Hull. These single-run results do not establish success probabilities or global optimality.

All five methods attain the Small Batch reference. Big-M and Hull complete in 0.55 and 0.57 s, respectively, compared with median DFO completion estimates ranging from 3.49 to 9.09 s. This demonstrates the competitiveness of algebraic methods when the formulation is available, although the different information access and timing protocols prevent direct speedup claims.

These single-run algebraic comparisons distinguish numerical feasibility, reference-target attainment, solver termination, and optimality certification. All six CSTR GLOA runs report `optimal` with numerically feasible solutions, but miss the IPOPT-table references, which are not independently certified global optima. Reevaluation of the archived GLOA and RIC vectors in the same original GDP, using its objective and variable-name mapping, gives a better RIC objective for every size. The gaps are 2.23–27.57% of the RIC values, approximately 2,228–27,571 times the comparison tolerance $10^{-5}\max\{1, |f_{\mathrm{RIC}}|\}$. Together with Column Big-M and CSTR30 Hull, eight raw `optimal` reports are target misses, with their original objectives, times, and termination labels retained. The six GLOA lower bounds equal their returned objectives and exceed the checked RIC values; Table 8 applies the same bound-consistency flag to all methods. The smaller Hull exceedances for CSTR5, CSTR10, and CSTR15 are only about 1.00, 1.81, and 1.54 times the comparison tolerance, respectively. These are numerical-consistency observations, not demonstrated software defects or evidence of general solver inferiority; the cause in GDPopt or BARON remains unresolved. LOA's time-limited CSTR15 run also returns an objective below the lookup reference, illustrating why target attainment and certification are assessed separately.

A separate CSTR15 check retained witness W03's discrete configuration (Supplementary Section S9) (SECQUOIA, 2026c). The returned objective was 2.8293940632857546, with maximum unscaled original-GDP violation 3.33×10^{-16}, passing a 10^{-8} residual target. The recorded GLOA lower bound still exceeds this objective by 0.6931579445, approximately 24,498 times the audit tolerance, so the numerical discrepancy persists at the tighter threshold without establishing exact feasibility or its cause. This additional check leaves the historical performance values and audit classifications unchanged.

Table 8: Full-process-GDP results: objective (measured completion seconds, including model-template preparation). GDPopt uses BARON for nonlinear subproblems and HiGHS for master problems; Big-M and Hull use BARON. Feasibility means numerical feasibility under the original-GDP residual test at 10^{-5}, not exact feasibility.

Model	RIC	GLOA	LOA	Big-M	Hull
CSTR 5	3.0619840 (31.27)	$3.1301981^{\dagger\ddagger}$ (3.76)	3.0619840 (31.82)	3.0620136 (9.12)	$3.0620146^{\dagger}$ (14.86)
CSTR 10	2.8895081 (214.06)	$3.4479640^{\dagger\ddagger}$ (4.56)	2.8895081 (234.13)	2.8894835 (123.74)	$2.8895358^{\dagger}$ (126.47)
CSTR 15	2.9660174^{TL} (1798.83)	$3.5225520^{\dagger\ddagger}$ (26.19)	2.8293506^{TL} (1799.38)	2.8293940^{TL} (1799.00)	$2.8293941^{\dagger}$ (940.36)
CSTR 20	2.9095042^{TL} (1799.04)	$3.6161230^{\dagger\ddagger}$ (6.79)	2.8895081^{TL} (1797.72)	2.7994063^{TL} (1798.72)	2.7994064^{TL} (1797.44)
CSTR 25	2.8594370^{TL} (1797.05)	$3.6478181^{\dagger\ddagger}$ (85.20)	2.7951055^{TL} (1797.46)	2.7814826^{TL} (1797.67)	2.7814826^{TL} (1797.41)
CSTR 30	2.8379523^{TL} (1792.52)	$3.1543081^{\dagger\ddagger}$ (718.80)	2.7844377^{TL} (1798.92)	2.7695698^{TL} (1797.31)	$2.8093862^{\dagger\ddagger}$ (1466.60)
Column	22271.549^{TL} (1797.54)	22271.549^{TL} (1798.15)	22271.549^{TL} (1797.77)	$19853.723^{\dagger\ddagger}$ (34.36)	20837.080^{TL} (1797.17)
Dynamic	-119.383^{TL} (1797.71)	-119.383^{TL} (1797.55)	-119.383^{TL} (1797.84)	-207.626^{TL} (1797.51)	-28.597^{TL} (1797.77)
Small Batch	167427.657 (12.86)	167427.657 (14.49)	167427.657 (5.11)	167427.657 (0.55)	167427.657 (0.57)

Raw termination is `optimal` unless TL denotes `maxTimeLimit` with a saved feasible incumbent. Reference-target classification uses the lookup-reference values in Table 1. ‡: reference target not attained despite the reported `optimal`; a better independently checked numerically feasible solution exists under the stated residual test, so the report is not treated as an independent optimality certificate. This is a target miss, not an execution failure. †: reported lower bound exceeds the best checked saved numerically feasible value f by more than $10^{-5}\max\{1, |f|\}$; the unrounded bound-consistency results and comparison solutions are provided in Supplementary Section S9, Tables S14–S15, and `algebraic_audit/table8_bound_audit.csv` in the companion data. Near-tolerance cases are discussed in the text. The same rule applies to all methods. All 45 saved vectors pass the original-GDP feasibility recheck at 10^{-5}, which does not establish optimality. Each method/model has one run; no restart probability or TTS99 is inferred. Times are full-run completion times.

4.4 Integrated Numerical Discussion

The comparison separates three sources of performance differences: which configurations a method visits, the evaluation costs of those configurations, and the work required to select them. The initialization, fallback, and stopping diagnostics describe the complete implementations without isolating the causal benefit of surrogate learning. RBFOpt reaches more Grid targets than the other compared methods at budgets of 30 evaluations and above. The prefix and uniform controls show that BAM's low-budget Grid advantage is explained by its fixed initial design, with additional gains from its later search at larger budgets. Because the Grid evaluations fix both original coordinates, their measured costs emphasize the evaluation interface; the process cases additionally expose variable NLP and preprocessing costs (see Supplementary Table S5 for model sizes). These distinctions limit extrapolation from either benchmark family alone.

The fixed-path point-cost scenarios in Supplementary Table S19 also change the ordering of median costs. At Grid B=100, RBFOpt costs 70.81 versus 64.42 s for BAM at the recorded scale, but 619.68 versus 640.12 s when saved evaluation costs are multiplied by ten. This scenario retains the recorded trajectories, search overhead and preparation cost; it neither predicts adaptation to different live time limits nor identifies a universal crossover.

Dynamic illustrates how budget and stopping behavior change completion-time rankings. RBFOpt reaches all ten targets at $B = 12$ with a 12.29 s median completion estimate, while DSDA first does so at $B = 30$ with 26.61 s. At the common budget $B = 30$, DSDA instead has the lower completion estimate, 26.61 versus 34.19 s. At that budget, DSDA uses a median of 23.5 callbacks (range 15–30), versus 30 in every RBFOpt run, with median post-target counts of 6.5 and 23, respectively (`manuscript/tables/run_records.csv`; Supplementary Section S10). These retrospective comparisons combine candidate selection and stopping behavior; they identify neither an out-of-sample budget policy nor first-target times within optimizer overhead.

The process results also depend on how unsuccessful configurations are handled. Logical rejection can be inexpensive, and early termination can reduce both evaluation count and total cost; such evaluations remain charged and all failed runs remain in the denominator. The CSTR results favor RBFOpt's lookup-target attainment under the stated size-dependent budgets, while DSDA performs well on Small Batch. The simultaneous changes in CSTR size, evaluation budget, and NLP initialization at CSTR30 do not isolate algorithmic scalability. The restricted Column box and the fixed local-solver initialization and acceptance policy further define the domain of these findings.

Algebraic formulations provide competitive reference results when supported, particularly for Small Batch. Their expression coverage and access to model structure differ from those of DFO, and the unresolved optimality reports discussed in Section 4.3 cannot serve as independent certificates.

Hit counts and completion-cost distributions carry the primary evidence, with empirical TTS99 providing a secondary restart summary.

5 Conclusions

This study compared four GDP configuration-search implementations with sampling controls under a shared evaluation protocol, separating target attainment, configuration-evaluation cost, and search overhead. Across the analytical and process benchmarks, higher target coverage did not consistently yield lower estimated full-run completion times. Initialization, fallback sampling, search computation, and stopping behavior help interpret these trade-offs, but their causal effects were not isolated.

The practical implication is to report target coverage and computational cost separately and to choose an evaluation budget together with the search method. Algebraic approaches remain useful alternatives when model expressions and suitable solvers are available.

The findings are conditional on finite candidate domains and frozen local-solver responses. Completion costs are retrospective estimates, not measured online optimizer–NLP times. Ten-seed hit rates do not establish 99% solution-time guarantees, and the conflicting algebraic optimality reports remain unresolved. Transfer to broader configuration spaces and live NLP evaluation remains to be tested.

6 Acknowledgement

The authors acknowledge and express sincere gratitude to the Davidson School of Chemical Engineering at Purdue University for providing a supportive research environment and funding.

Data and code availability

The code, frozen evaluation data, archived execution records, and analysis scripts are maintained in `SECQUOIA/Black-Box-for-GDP`; the repository remains private at this revision and public release is planned after submission, subject to the repository administrator enabling access (SECQUOIA, 2026a,b). The additional comparisons are identified by commit `6e083eb49b497ce4fea90131e18cc0459db682e8` on `review/additional-comparisons`. Supplementary Section S8 links the selected Grid and Process datasets, source manifests, and regeneration instructions. The original execution revisions and source hashes are retained separately from this repository snapshot.

References

Bergamini, M.L., Aguirre, P., Grossmann, I., 2005. Logic-based outer approximation for globally optimal synthesis of process networks. Comput. Chem. Eng. 29, 1914–1933. doi:10.1016/j.compchemeng.2005.04.003.

Chen, Q., Johnson, E.S., Bernal, D.E., Valentin, R., Kale, S., Bates, J., Siirola, J.D., Grossmann, I.E., 2022. Pyomo.GDP: an ecosystem for logic based modeling and optimization development. Optim. Eng. 23, 607–642. doi:10.1007/s11081-021-09601-7.

Costa, A., Nannicini, G., 2018. RBFOpt: an open-source library for black-box optimization with costly function evaluations. Math. Program. Comput. 10, 597–629. doi:10.1007/s12532-018-0144-7.

Cozad, A., Sahinidis, N.V., Miller, D.C., 2014. Learning surrogate models for simulation-based optimization. AIChE J. 60, 2211–2227. doi:10.1002/aic.14418.

D-Wave Systems, n.d. dwave-samplers: Classical algorithms for solving binary quadratic models. URL: https://github.com/dwavesystems/dwave-samplers/releases/tag/1.6.0. Version 1.6.0; classical CPU simulated annealing; accessed October 15, 2026.

Dolan, E.D., Moré, J.J., 2002. Benchmarking optimization software with performance profiles. Math. Program. 91, 201–213. doi:10.1007/s101070100263.

Grossmann, I.E., Lee, S., 2003. Generalized convex disjunctive programming: Nonlinear convex hull relaxation. Comput. Optim. Appl. 26, 83–100. doi:10.1023/A:1025154322278.

Kirkpatrick, S., Gelatt, Jr., C.D., Vecchi, M.P., 1983. Optimization by simulated annealing. Science 220, 671–680. doi:10.1126/science.220.4598.671.

Kitai, K., Guo, J., Ju, S., Tanaka, S., Tsuda, K., Shiomi, J., Tamura, R., 2020. Designing metamaterials with quantum annealing and factorization machines. Phys. Rev. Res. 2, 013319. doi:10.1103/PhysRevResearch.2.013319.

Larson, J., Menickelly, M., Wild, S.M., 2019. Derivative-free optimization methods. Acta Numerica 28, 287–404. doi:10.1017/S0962492919000060.

Ma, K., Rios, L.M., Bhosekar, A., Sahinidis, N.V., Rajagopalan, S., 2023. Branch-and-Model: a derivative-free global optimization algorithm. Comput. Optim. Appl. 85, 337–367. doi:10.1007/s10589-023-00466-3.

Moré, J.J., Wild, S.M., 2009. Benchmarking derivative-free optimization algorithms. SIAM J. Optim. 20, 172–191. doi:10.1137/080724083.

Nannicini, G., 2021. On the implementation of a global optimization method for mixed-variable problems. Open J. Math. Optim. 2, 1–25. doi:10.5802/ojmo.3.

Ovalle, D., Liñán, D.A., Lee, A., Gómez, J.M., Ricardez-Sandoval, L., Grossmann, I.E., Bernal Neira, D.E., 2025. Logic-Based Discrete-Steepest Descent: A solution method for process synthesis Generalized Disjunctive Programs. Comput. Chem. Eng. 195, 108993. doi:10.1016/j.compchemeng.2024.108993.

Peng, Z., Lee, A., Bernal Neira, D.E., 2024. Addressing discrete dynamic optimization via a logic-based discrete-steepest descent algorithm, in: 2024 IEEE 63rd Conference on Decision and Control (CDC), IEEE. pp. 1664–1669. doi:10.1109/CDC56724.2024.10886477.

Pyomo Development Team, n.d. GDPopt logic-based solver. URL: https://pyomo.readthedocs.io/en/6.10.0/explanation/solvers/gdpopt.html. Version 6.10.0; accessed October 20, 2026.

Raman, R., Grossmann, I.E., 1994. Modelling and computational techniques for logic based integer programming. Comput. Chem. Eng. 18, 563–578. doi:10.1016/0098-1354(93)E0010-7.

Regis, R.G., Shoemaker, C.A., 2007. A stochastic radial basis function method for the global optimization of expensive functions. INFORMS J. Comput. 19, 497–509. doi:10.1287/ijoc.1060.0182.

Rendle, S., 2010. Factorization machines, in: 2010 IEEE International Conference on Data Mining, pp. 995–1000. doi:10.1109/ICDM.2010.127.

Sahinidis, N.V., 1996. BARON: A general purpose global optimization software package. J. Glob. Optim. 8, 201–205. doi:10.1007/BF00138693.

SECQUOIA, 2026a. Black-Box-for-GDP: additional genetic-search and sampling comparisons. URL: https://github.com/SECQUOIA/Black-Box-for-GDP/tree/6e083eb49b497ce4fea90131e18cc0459db682e8/results/review_comparison_20261020. Analysis snapshot on review/additional-comparisons, accessed October 20, 2026; original execution provider 38d917ef75ef6767c2a9f238d88be19bf0b89160; private pending administrator-enabled public release.

SECQUOIA, 2026b. Black-Box-for-GDP: analytical Grid-GDP and process benchmark archive. URL: https://github.com/SECQUOIA/Black-Box-for-GDP/tree/cb70771b2037301bb6d0e38a2362d49b53fcda7d. Snapshot of main; commit cb70771b2037301bb6d0e38a2362d49b53fcda7d, accessed October 19, 2026.

SECQUOIA, 2026c. Black-Box-for-GDP: CSTR15 fixed-Boolean identity preprocessing and tightened-feasibility validation. URL: https://github.com/SECQUOIA/Black-Box-for-GDP/tree/854a1a7fc5a20a5489d82ef7991a16aaf89025f3/results/review_feasibility_tautology_20261020. Additional numerical validation of the archived W03 configuration, accessed October 20, 2026; original benchmark records and execution revisions preserved; private pending public release.

SECQUOIA, n.d. bbo_via_fmqa: Black-box optimization via factorization machines and quantum annealing. URL: https://github.com/SECQUOIA/bbo_via_fmqa/tree/eb3a20d4b5a3773d17ff31d72a35d51f58e84e56. QHD_Data branch; commit eb3a20d4b5a3773d17ff31d72a35d51f58e84e56; upstream PyTorch FM and BQM code; accessed October 19, 2026.

The Optimization Firm, 2026. BAM python interface documentation: bam-solver 5.0.0. URL: https://pypi.org/project/bam-solver/5.0.0/. Version 5.0.0, released September 11, 2026; executed engine 2026.9.10; accessed October 20, 2026.

Türkay, M., Grossmann, I.E., 1996. Logic-based MINLP algorithms for the optimal synthesis of process networks. Comput. Chem. Eng. 20, 959–978. doi:10.1016/0098-1354(95)00219-7.

Wu Group, n.d. Quantum Hamiltonian descent: 2D test functions. https://jiaqileng.github.io/quantum-hamiltonian-descent/nonconvex2d.html. Source collection for the 21 adapted finite grids used here; accessed September 29, 2026.